

.LIB File

Monday, September 14, 2026 5:27 PM

**From RTL to Silicon
It All Starts with
Technology Data**

What is a .lib File ?

The Technology Knowledge Base for EDA Tools

Same RTL,
Different Technology,
Different Speed !

1 What is a .lib file ?

A .lib file is a pre-characterized model of every standard cell (and sometimes macros) in a technology, stored in **ASCII text format** using the **Liberty format**. It describes the **timing, power, functionality and electrical characteristics** of each cell under specific **Process, Voltage and Temperature (PVT)** conditions.

Example (snippet from .lib):

```
library (slow_0p72v_125c) {
  delay_model : table_lookup ;
  time_unit : "1ps" ;
  voltage_unit : "1V" ;
  current_unit : "1uA" ;
  capacitive_load_unit (1, ff) ;
  ...
}
```

2 What is the full form of .lib ?

.lib = Liberty Timing File

- Liberty** is the standard format developed by Synopsys and now widely adopted across the industry (Cadence, Synopsys, Foundries, etc.).
- It is also called a **Liberty Library** or **Liberty Timing Library**.

Key Points:

- Industry standard format (IEEE 1801/Liberty).
- Used by all major EDA tools.
- Human-readable ASCII format.
- Contains timing, power, and constraint information.

3 Why is a .lib file required in ASIC design ?

- EDA tools (Genus, DC, Innovus, Tempus, etc.) need to know how each cell behaves in real silicon.
- It provides delay, transition, power, area and electrical limits for each cell.
- Helps in technology mapping, timing analysis, power analysis, placement planning and design rule checking.
- Without .lib, tools cannot estimate timing, power or verify design constraints.

Same RTL + Different .lib = Different timing, power and area results

4 Who provides the .lib file ?

- The .lib file is provided by the semiconductor **foundry or technology provider** (e.g., TSMC, Samsung, Intel, GlobalFoundries) as part of the technology development kit (PDK).
- Generated using **characterization tools** like **Cadence Liberate** or **Synopsys SiliconSmart** after detailed SPICE simulations.
- It is delivered for each process node, PVT corner, and sometimes for different Vt (LVT/SVT/HVT) and drive strength variants.

Foundry provides characterized .lib as part of the PDK

5 Where does the information inside .lib come from ?

- Foundry designs each standard cell at the transistor level.
- They run detailed SPICE simulations across many input slews, output loads and PVT conditions.
- From these simulations, they measure:
 - propagation delays
 - output transition (slew)
 - power (internal, switching, leakage)
 - setup/hold times (for sequential cells)
 - electrical limits (max cap, max transition, max fanout)
 - cell area, etc.
- These measured values are stored in .lib in tabular form (lookup tables).

6 What does pre-characterized mean ?

- It means the cells are already simulated and measured by the foundry **before** you use them.
- All timing, power and electrical values are pre-computed and stored in the .lib file.
- You do not need to run transistor-level simulations yourself. EDA tools simply look up the required values from the .lib file.

Analogy: Restaurant Menu

You don't negotiate the price every time you order. You just look up the menu.

Similarly, .lib is the menu and EDA tools (Genus/Innovus) just look up the values (delays, power, etc.) for each cell.

7 Why can't Genus or Innovus know cell timing without .lib ?

- Genus, Innovus or any EDA tool is a software program. It has never seen the real silicon.
- It does not know how fast a transistor switches in a specific technology (e.g., TSMC 7 nm).
- The same RTL can have completely different timing and power in different technologies.
- Only the foundry, through SPICE simulations, can tell the real delay, power and electrical behavior of each cell. This information is stored in the .lib file and used by the tools.

Same RTL code (Adder)
`result <= a + b;`

TSMC 7 nm .lib
 Delay = 50 ps
 Power = 1 μW

TSMC 28 nm .lib
 Delay = 120 ps
 Power = 3 μW

EDA tools need .lib to know these values. Without .lib, they have no idea about real silicon behavior !

Key Takeaway: .lib is the technology knowledge base for EDA tools. It contains pre-characterized timing, power, area and electrical data for each cell, provided by the foundry, and is essential for synthesis, STA and physical design.

References:

- Synopsys Liberty User Guide (liberty.pdf)
- Cadence Liberate Documentation
- TSMC / Foundry PDK Documentation
- Your shared PDF: ".LIB file.pdf" (Pages 1-5)

**"No .lib, No Timing
No Silicon !"**

**Timing
Power
Functionality
Physical**

What Information Does a .lib File Contain ?

Complete View of Standard Cell Information in Liberty File

Technology Data
Drives Better Design !

1 Overview: Information for Every Standard Cell

- A .lib file contains pre-characterized information for every standard cell (and macro) in the technology.
- It provides **timing, power, functionality and electrical limits** under specific PVT conditions.
- EDA tools (Genus, DC, Innovus, Tempus, etc.) use this information for synthesis, STA, placement, CTS and power analysis.

Main information in .lib :

- Cell functionality (Boolean function)
- Cell delay (timing tables)
- Output transition (slew)
- Power information (internal, switching, leakage)
- Electrical limits (max cap, max transition, max fanout)
- Cell area (for placement)
- Sequential timing (setup, hold, clock-to-Q)
- Cell attributes (drive strength, Vt, etc.)

2 Cell Functionality

What logic does the cell implement ?

- Defines the **Boolean function** of the cell.
- Helps synthesis tool map RTL logic to the correct standard cell.
- Specified using the **function** attribute in the pin block (usually for the output pin).

Example: 2-input AND gate (AND2_X1)

```
cell (AND2_X1) {
  ...
  pin (Y) {
    direction : output;
    function : "A & B";
  }
}
```

Y = A & B

Functionality is technology independent, but represented in .lib for tool mapping.

3 Cell Delay

How fast does the cell switch ?

- Propagation delay from input to output.
- Depends on input transition (slew) and output load (capacitance).
- Different values for rise delay and fall delay.
- Stored in lookup tables (LUTs) inside the **timing** block.

Example: cell_rise and cell_fall tables

```
timing () {
  related_pin : "A";
  timing_type : combinational;
  cell_rise ("delay_template") {
    values ("0.01, 0.02, 0.03",...);
  }
  cell_fall ("delay_template") {
    values ("0.015, 0.028, 0.045",...);
  }
}
```

Tool interpolates the table to get delay for actual slew and load.

4 Transition or Slew

How slow does the output transition ?

- Output transition time (slew) at the output pin.
- Depends on input slew and output load.
- Defines how fast the output waveform rises or falls (usually 10% to 90%).
- Stored in `rise_transition` and `fall_transition` tables.

5 Power Information

How much power does the cell consume ?

- Dynamic power per transition. (internal + switching)
- Internal power: power consumed inside the cell.
- Switching power: power to charge/discharge the load.
- Specified using `internal_power` and `switching_power` tables.

6 Leakage Power

How much power does it waste when idle ?

- Leakage power exists even when the cell is not switching.
- Caused by sub-threshold leakage, gate leakage, junction leakage.
- Specified using `leakage_power` attribute.
- Can be state-dependent (using when condition).

Example: rise_transition table

```
timing () {
  related_pin : "A";
  rise_transition ("delay_template") {
    values ("5, 8, 12",...);
  }
}
```

Slew affects the delay of the next stage.

Example: internal power table

```
internal_power () {
  related_pin : "A";
  rise_power ("power_template") {
    values ("0.1, 0.2, 0.35",...);
  }
  fall_power ("power_template") {
    values ("0.08, 0.18, 0.32",...);
  }
}
```

Total Dynamic Power:
 $P_{dyn} = P_{internal} + P_{switching}$
 Used by power analysis tools (e.g., Cadence Voltus, Synopsys PrimePower).

example: leakage power

```
leakage_power () {
  value : 1.2e-06; // 1.2 μW
}
// State dependent leakage
leakage_power () {
  value : 8e-07;
  when : "A == 0 & B == 1";
}
```

Leakage depends on cell type and threshold voltage (LVT/SVT/HVT).
 Important for low-power design and total power estimation.

7 Electrical Limits
 What are the cell's usage limits?

- Defines the maximum allowed load, transition and fanout for reliable operation.
- Used by synthesis and STA to check and fix design rule violations (DRVs).

Example: electrical limits

```
pin (Y) {
  direction : output;
  max_capacitance : 50.0; // fF
  max_transition : 100.0; // ps
  max_fanout : 20;
}
```

If these limits are violated, tools insert buffers, resize cells or restructure the logic.

8 Cell Area
 How big is the cell?

- Physical area of the cell in μm^2 .
- Used by placement tools to calculate utilization and for area (gate count) reporting.
- Multiple variants (X1, X2, X4, LVT/SVT/HVT) have different areas.

Example: cell area

```
cell (NAND2_X1) {
  area : 0.525; // in μm²
  cell_leakage_power : 1.2e-06;
  ...
}
```

Different drive strengths:

Cell	Area (μm^2)
NAND2_X1	0.525
NAND2_X2	0.910
NAND2_X4	1.780

9 Timing Constraints (Sequential Cells)
 Setup, Hold and Clock-to-Q timing

- For sequential cells (flip-flops, latches), .lib contains:
 - Setup time (data must be stable before clock)
 - Hold time (data must be stable after clock)
 - Clock-to-Q delay (from clock to output)
 - Recovery and removal (for async reset/set)

Example: setup/hold and clock-to-Q

```
timing () {
  related_pin : "CLK";
  timing_type : setup_rising;
  ...
}
timing () {
  related_pin : "CLK";
  timing_type : hold_rising;
  ...
}
timing () {
  related_pin : "CLK";
  timing_type : clock_rising;
  ...
}
```

These timing constraints are critical for accurate STA and timing closure.

Key Takeaway: A .lib file contains pre-characterized timing, power, functionality, area and electrical limits for every standard cell. It is the technology data book for EDA tools and is essential for synthesis, STA, power analysis and physical design.

References:
 1. Synopsys Liberty User Guide (liberty.pdf)
 2. Cadence Liberate Documentation
 3. TSMC / Foundry PDK Documentation
 4. Your shared PDF: ".lib file.pdf"

"Accurate .lib Builds Reliable Silicon."

What Information .lib File Contain ?

Complete View of Standard Cell Information in Liberty (.lib) File

1 What information does .lib provide for every cell ?

- A .lib file contains pre-characterized information for every standard cell (and macro) in the technology.
- It provides **timing, power, functionality and electrical limits** under specific PVT conditions.

For every standard cell, .lib provides:

- Cell functionality (Boolean function)
- Cell delay (timing tables)
- Output transition (slew)
- Power information (internal, switching, leakage)
- Electrical limits (max cap, max transition, max fanout)
- Cell area (for placement)
- Sequential timing (setup, hold, clock-to-Q)
- Cell attributes (drive strength, Vt, etc.)

2 What is cell functionality and cell delay ?

Cell Functionality

- Defines the **Boolean** function of the cell.
- Helps synthesis tool map RTL logic to the correct standard cell.
- Specified using the **function** attribute in the pin block (usually for the output pin).

Example: 2-input AND gate (AND2_X1)

```
cell (AND2_X1) {
  pin (Y) {
    direction : output;
    function : "A & B";
  }
}
```

Cell Delay

- Propagation delay from input to output.
- Depends on input transition (slew) and output load (capacitance).
- Stored in lookup tables (LUTs) inside the timing block.

3 What is slew or transition ?

- Slew** (transition) is the output transition time at the output pin.
- Depends on input slew and output load.
- Defines how fast the output waveform rises or falls (usually 10% to 90%).
- Stored in **rise_transition** and **fall_transition** tables.

Example: rise_transition table

```
timing () {
  related_pin : "A";
  rise_transition ("delay_template") {
    values ("5, 8, 12", ...);
  }
}
```

Slew affects the delay of the next stage.

4 What is power information and leakage power ?

Power Information

- Dynamic power per transition (internal + switching).
- Internal power: power consumed inside the cell.
- Switching power: power to charge/discharge the load.
- Specified using **internal_power** and **switching_power** tables.

Example: internal power table

```
internal_power () {
  related_pin : "A";
  rise_power ("power_template") {
    values ("0.1, 0.2, 0.35", ...);
  }
  fall_power ("power_template") {
    values ("0.08, 0.18, 0.32", ...);
  }
}
```

Total Dynamic Power:
 $P_{dyn} = P_{internal} + P_{switching}$
 Used by power analysis tools (e.g., Cadence Voltus, Synopsys PrimePower).

Leakage Power

- Leakage power exists even when the cell is not switching.
- Caused by sub-threshold leakage, gate leakage, junction leakage.
- Specified using **leakage_power** attribute.
- Can be state-dependent (using when condition).

5 What are electrical limits ?

- Defines the maximum allowed load, transition and fanout for reliable operation.
- Used by synthesis and STA to check and fix design rule violations (DRVs).

Example: electrical limits

```
pin (Y) {
  direction : output;
  max_capacitance : 50.0; // fF
  max_transition : 100.0; // ps
  max_fanout : 20;
}
```

Parameter	Meaning	Unit (Typical)
max_capacitance	Maximum load allowed at the output pin	fF (or pF)
max_transition	Maximum output transition time	ps
max_fanout	Maximum number of loads that can be driven	- (count)

If these limits are violated, tools insert buffers, resize cells or restructure the logic.

6 What are timing constraints and cell area ?

Timing Constraints (Sequential Cells)

- For sequential cells (flip-flops, latches), .lib contains:
 - Setup time (data must be stable before clock)
 - Hold time (data must be stable after clock)
 - Clock-to-Q delay (from clock to output)
 - Recovery and removal (for async reset/set)
- Specified using **timing_type** (e.g., setup_rising, hold_rising, clock_rising).

Example: setup timing for DFF

```
timing () {
  related_pin : "CLK";
  timing_type : setup_rising;
  ...
}
```

Cell Area

- Physical area of the cell in μm^2 .
- Used by placement tools to calculate utilization and for area (gate count) reporting.
- Multiple variants (X1, X2, X4, LVT/SVT/HVT) have different areas.

Example: cell area

```
cell (NAND2_X1) {
  area : 0.525; // in μm²
  cell_leakage_power : 1.2e-06;
  ...
}
```

Different drive strengths:

Cell	Area (μm^2)
NAND2_X1	0.525
NAND2_X2	0.910
NAND2_X4	1.780

Key Takeaway: .lib file is the technology data book for EDA tools. It contains functionality, timing (delay & transition), power (internal & leakage), electrical limits, timing constraints and cell area for every standard cell.

References:
 1. Synopsys Liberty User Guide (liberty.pdf)
 2. Cadence Liberate Documentation
 3. TSMC / Foundry PDK Documentation
 4. Your shared PDF: ".lib file.pdf"

"Accurate .lib Builds Reliable Silicon."

Complete Anatomy of a .lib File

Understanding the Structure of Liberty (Timing) File

1 What is the basic structure of a .lib file ?

- A .lib file is written in **Liberty format** (ASCII text).
- It has a **hierarchical** structure made of multiple blocks.
- Each block describes specific information about the technology, cells, pins, timing, power and constraints.
- The general structure is:

```
library (LIB_NAME) {
  operating_conditions
  wire_load (optional)
  ...
}
```

2 What is a library block ?

- The library block is the top-level block in a .lib file.
- It defines the overall library attributes, units, delay model, operating conditions, and technology information.
- It contains one or more cell blocks.

```
library (s1ow_0p72v_125c) {
  delay_model : table_lookup ;
  time_unit : "1ps";
  voltage_unit : "1V";
}
```

Library information includes:

- Library name
- Units (time, voltage)

5 How are these blocks connected ?

- The library block contains multiple cell blocks.
- Each cell block contains multiple pin blocks.
- Each pin block can have timing, power and other information.
- All information is hierarchically organized from general (library) to specific (cell → pin → timing).

Library
 (library attributes, units, PVT, etc.)

```

lu_table_template
cell (CELL_NAME) {
  cell attributes
  pin (PIN_NAME) {
    pin attributes
    timing()
    internal_power()
    leakage_power()
  }
}
}

```

```

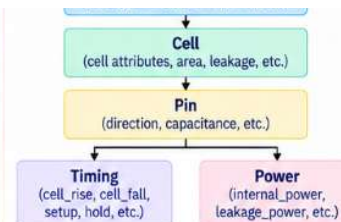
current_unit : "1uA" ;
capacitive_load_unit (1, ff) ;

operating_conditions(slow_0p72v_125c) {
  process : 1.0 ;
  voltage : 0.72 ;
  temperature : 125.0 ;
}

default_operating_conditions : slow_0p72v_125c ;
...

```

- Delay model (NLD/CCS)
- Operating conditions (PVT)
- Wire load model (optional)
- Other global attributes



3 What is a cell block and pin block ?

Cell Block

- The cell block defines a single standard cell.
- It contains cell name, area, leakage power, and all its pins (pin blocks).
- Each standard cell (e.g., NAND2_X1, DFF_X1) has its own cell block.

```

cell (NAND2_X1) {
  area : 0.525 ;
  cell_leakage_power : 1.2e-06 ;
  ...
  pin (A) { ... }
  pin (B) { ... }
  pin (Y) { ... }
}

```

Cell information includes:

- Cell name
- Area
- Leakage power
- Cell attributes
- Pin blocks

Pin Block

- The pin block defines information about each pin of the cell.
- It contains pin name, direction, capacitance, function, and timing/power blocks.

```

pin (A) {
  direction : input ;
  capacitance : 0.0023 ;
  function : "A" ;
  timing() { ... }
  internal_power() { ... }
  leakage_power() { ... }
}

```

Pin information includes:

- Pin name
- Direction (input/output)
- Capacitance
- Function
- Timing block
- Power block

4 What is a timing block and power block ?

Timing Block

- The timing block defines the timing behavior between a related pin and the current pin.
- It contains cell_rise, cell_fall, rise_transition, fall_transition tables for combinational cells.
- For sequential cells, it contains setup, hold and clock-to-Q timing tables.

```

pin (Y) {
  direction : output ;
  timing() {
    related_pin : "A" ;
    timing_type : combinational ;
    cell_rise (delay_template) { values { ... } ; }
    cell_fall (delay_template) { values { ... } ; }
    rise_transition (delay_template) { values { ... } ; }
    fall_transition (delay_template) { values { ... } ; }
  }
}

```

Timing block includes:

- related_pin
- timing_type
- cell_rise / cell_fall
- rise_transition / fall_transition
- setup / hold (for FFs)
- clock-to-Q (for FFs)
- timing_sense, etc.

Power Block

- The power block defines power consumption of the cell.
- It contains internal_power (dynamic power) and leakage_power tables.

```

pin (Y) {
  direction : output ;
  internal_power() {
    related_pin : "A" ;
    rise_power (power_template) { values { ... } ; }
    fall_power (power_template) { values { ... } ; }
  }
  leakage_power() {
    value : 1.2e-07 ;
    when : "A & B" ;
  }
}

```

Power block includes:

- internal_power (rise_power, fall_power)
- leakage_power
- power tables (LUTs)
- power related_pin
- state-dependent leakage (when condition)



Key Takeaway:

A .lib file is hierarchical: **library** → **cell** → **pin** → **timing/power**. It contains all the pre-characterized information (timing, power, functionality, area, constraints) for each standard cell.



- References:
1. Synopsys Liberty User Guide (liberty.pdf)
 2. Cadence Liberate Documentation
 3. TSMC / Foundry PDK Documentation
 4. Your shared PDF: "LIB file.pdf"

"Library Data Builds Real Chips"



Liberty File
Timing | Power |
Functionality | PVT

.lib Header and Library Attributes

Understanding the Top-Level Information in a Liberty File

Right Library
Right Results
Real Silicon!

1 What information is present in the .lib header ?

- The header contains general information about the library such as:
 - Library name
 - Technology node
 - PVT corner (process, voltage, temperature)
 - Tool used to generate the library (e.g., Cadence Liberate)
 - Date / version
 - Units (time, voltage, current, capacitance)
 - Delay model (NLD/CCS)

Example: .lib header

```

/* ===== */
Library      : slow_0p72v_125c
Technology   : 7nm FinFET
Corner       : Slow Process, 0.72V, 125C
Generated by : Cadence Liberate
Date        : 2024-05-12
===== */

library (slow_0p72v_125c) {
  delay_model : table_lookup ;
  time_unit   : "1ps" ;
  voltage_unit : "1V" ;
  current_unit : "1uA" ;
  capacitive_load_unit (1, ff) ;
  ...
}

```

2 What is delay_model ?

- **delay_model** specifies how the delay, transition and power values are represented in the library.
- Two common types:

NLDM
(Non-Linear
Delay Model)

- Most commonly used in synthesis.
- Uses 2D lookup tables for delay and transition.
- Simple and compact.

CCS
(Current
Source Model)

- More accurate model.
- Uses current source waveforms instead of simple tables.
- Commonly used for signoff and advanced nodes.

Example:

```

delay_model : table_lookup ; // NLDM
// or
delay_model : ccs ; // CCS

```

3 What are time units and voltage units ?

- **time_unit** defines the unit for all timing values in the .lib file (e.g., cell delay, transition, setup/hold).
- Typically specified in picoseconds (ps).
- **voltage_unit** defines the unit for voltage values (e.g., operating voltage, threshold).
- Typically specified in volts (V).

Example:

```

time_unit   : "1ps" ;
voltage_unit : "1V" ;

```

Common units used

Quantity	Typical Unit
Time	1ps, 1ns
Voltage	1V

All timing, power and electrical values are scaled according to these units.

4 What are current units and capacitance unit ?

- **current_unit** defines the unit for current values (e.g., internal current used in power calculation).
- Typically specified in microampere (uA) or milliampere (mA).
- **capacitive_load_unit** defines the unit for capacitance used in load and pin capacitance values.
- Typically specified in femtofarad (ff) or picofarad (pF).

Example:

```

current_unit      : "1uA" ;
capacitive_load_unit (1, ff) ;

```

Common units used

Quantity	Typical Unit
Current	1uA, 1mA
Capacitance	1ff, 1pF

Capacitance unit is specified as a pair (scale, unit) e.g., (1, ff) means 1 femtofarad.

5 Why must units be checked ?

- All numerical values in the .lib file (delay, transition, power, capacitance, etc.) are associated with these units.
- If the tool interprets the wrong units, all values will be scaled incorrectly.
- This can lead to wrong timing, power and DRV analysis in synthesis, place & route, and STA.

Example:

If time_unit is "1ns" instead of "1ps", a delay value of 100 will be interpreted as:

100 ns (wrong) instead of **100 ps (correct)**

Always verify the units before using a library in Genus / Innovus / Tempus.

6 What happens if units are incorrect ?

- Timing values may be 1000× higher or lower (e.g., ps vs ns).
- Power reports will be incorrect (uA vs mA).
- Capacitance and load values will be misinterpreted (ff vs pF).
- Synthesis may make wrong optimization decisions.
- Can lead to timing failures, power estimation errors, and even silicon failure.

Example Scenario:

- Library has time_unit = "1ns" (wrongly loaded).
- Tool thinks 50 (from table) = 50 ns instead of 50 ps.
- Design appears to meet timing easily in reports, but actual silicon will be 1000× slower → design fails.



Loading the wrong library or wrong units can make everything look fine in reports, but silicon will fail.



Key Takeaway:

The .lib header defines the library name, PVT, tool, units and delay model. Always verify units (time, voltage, current, capacitance) to avoid incorrect timing, power and silicon failures.



- References:
1. Synopsys Liberty User Guide (liberty.pdf)
 2. Cadence Liberate Documentation
 3. TSMC / Foundry PDK Documentation
 4. Your shared PDF: "LIB file.pdf"

"Correct Data Leads to Correct Silicon."



Timing
Power
Physical Design
Standard Cells

Understanding the cell() Block in .lib File

Cell Level Information in Liberty (Timing) Format

Each Cell
Has a Story

1 What is a cell block ?

- The **cell** block defines a single standard cell in the .lib file.
- It contains all the information about the cell such as cell name, area, leakage power, cell attributes and its pin information (inside pin blocks).
- Each standard cell (e.g., NAND2_X1, DFF_X2) has its own cell block.
- It is one of the major blocks inside the library block.

Example: cell block

```
cell (NAND2_X1) {
  area : 0.525 ;
  cell_leakage_power : 1.2e-06 ;

  pin (A) { ... }
  pin (B) { ... }
  pin (Y) { ... }
}
```

2 How does .lib define a standard cell ?

- A **standard cell** is defined using the keyword **cell (cell_name) { ... }**.
- Inside the cell block, various attributes are specified such as area, leakage power, cell attributes, and pin blocks.
- Each pin of the cell (input, output, etc.) is defined using a **pin** block inside the cell block.

Example: NAND2_X1 cell definition (simplified)

```
cell (NAND2_X1) {
  area : 0.525 ;
  cell_leakage_power : 1.2e-06 ;
  cell_footprint : "NAND2" ;
  cell_leakage_power : 1.2e-06 ;

  pin (A) { direction : input; ... }
  pin (B) { direction : input; ... }
  pin (Y) { direction : output; ... }
}
```

3 What is cell name, area and leakage power ?

Attribute	Description
Cell name	- Unique name of the standard cell. - Follows a naming convention (e.g., NAND2_X1, INV_X2, DFF_X1). - Indicates function, drive strength and VT type.
Cell area	- Physical area of the cell in μm^2. - Used by placement tools for utilization and density calculation. - Given by area attribute.
Leakage power	- Static power consumed by the cell when it is not switching. - Specified by cell_leakage_power attribute. - Typically given in Watts (W).

Example:

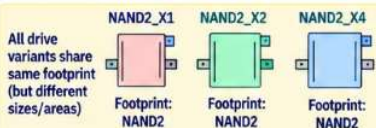
```
cell (NAND2_X1) {
  area : 0.525 ; // in  $\mu\text{m}^2$ 
  cell_leakage_power : 1.2e-06 ; // in W
}
```

4 What is cell footprint ?

- Cell footprint** specifies the physical layout template name of the cell.
- It indicates the base layout / physical shape used for this cell.
- Multiple drive strengths (X1, X2, X4) of the same function may share the same footprint.
- Used by physical design tools (e.g., Innovus) to match with LEF and for cell placement.

Example:

```
cell (NAND2_X1) {
  cell_footprint : "NAND2" ;
}
```



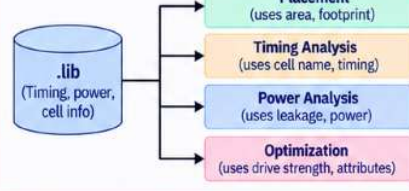
All drive variants share same footprint (but different sizes/areas)

Footprint: NAND2 Footprint: NAND2 Footprint: NAND2

5 How does a PD tool use cell information ?

- Placement** : Uses cell area and footprint for placement and utilization calculation.
- Timing** : Uses cell name to match with timing data, pin information and timing arcs.
- Power** : Uses leakage power for power analysis
- and IR drop estimation.
- Physical consistency** : Matches cell name and footprint with LEF file.
- Optimization** : Helps in resizing, buffering and drive strength selection.

How PD tools use cell information



Monte Carlo: stochastic stages.

Used across different stages:

- ✓ Synthesis (Genus)
- ✓ Placement (Innovus)
- ✓ STA (Tempus)
- ✓ Power Analysis (Joules/Voltus)
- ✓ Physical Verification (LEF/LIB check)

Key Takeaway: The cell() block in a .lib file defines each standard cell with its name, area, leakage power, footprint and pins. This information is used by PD tools for placement, timing, power analysis and optimization.

References:

- Synopsys Liberty User Guide (liberty.pdf)
- Cadence Innovus User Guide
- Cadence Genus Synthesis User Guide
- Your shared PDF: "LIB file.pdf"

"Library Defines the Cell, Tools Build the SoC!"



Standard Cells
Drive Strength
Threshold Voltage
ASIC Design

Cell Names, Drive Strength and Vt

Understanding Standard Cell Naming Convention in .lib Files

Different Choices, Optimized SoC!

1 How do you read a standard cell name ?

- A **standard cell name** gives information about function, drive strength and threshold voltage.
- The naming convention may vary by foundry, but a common format is:

Function _ DriveStrength _ Vt _ AdditionalInfo

Example:

NAND2

Function
(NAND with 2 inputs)

X1

Drive strength

SVT

Threshold voltage type

2 What is drive strength ?

- Drive strength** indicates the current driving capability of a cell, i.e., how much load (capacitance) the cell can drive.
- Higher drive strength means a stronger (wider) transistor, which can drive larger loads and provides faster switching, but at the cost of higher area and power.
- Drive strength is usually indicated by X1, X2, X4, X8, etc.

Conceptual View

X1



Small size
Drives small load

X2



2x size
Drives ~2x load

X4



4x size
Drives ~4x load

3 What is X1, X2 and X4 ?

- X1, X2, X4 are different drive strength variants of the same cell.
- X1 is the minimum (unit) drive strength.
- X2 has approximately 2x the drive capability of X1.
- X4 has approximately 4x the drive capability of X1.
- Larger drive strengths (X8, X16, ...) are also available for higher load requirements.

Cell Name	Relative Drive Strength	Typical Use Case
NAND2_X1	1x	Small load, low power
NAND2_X2	2x	Medium load
NAND2_X4	4x	Large load, critical path
NAND2_X8	8x	Very large load

4 What is LVT & SVT & HVT ?

- Vt** stands for threshold voltage of the transistors inside the cell.
- Standard cells are available in different Vt flavors to trade-off between performance and leakage power.
- Common types:

Vt Type	Full Form	Characteristics
LVT	Low Threshold Voltage	- Lowest Vt - Fastest timing

5 How do drive strength and Vt affect timing and power ?

- Higher drive strength** (e.g., X4) → lower delay (faster), but higher dynamic power and larger area.
- Lower drive strength** (e.g., X1) → higher delay (slower), but lower power and smaller area.
- Lower Vt (LVT)** → faster timing, but higher leakage power.
- Higher Vt (HVT)** → slower timing, but lower leakage power.

Parameter	Effect on Timing	Effect on Power	Effect on Area
Higher Drive Strength	Lower delay	Higher dynamic power	Larger area
Lower Drive Strength	Higher delay	Lower power	Smaller area
Lower Vt (LVT)	Faster timing	Higher leakage power	-
Higher Vt (HVT)	Slower timing	Lower leakage power	-

6 How does synthesis select different cells ?

- The synthesis tool (e.g., Cadence Genus) uses the .lib file, design constraints (timing, power, area) and the target library to choose the best cell (drive strength and Vt) for each instance.
- For critical paths, it may choose higher drive strength and/or LVT cells to meet timing.
- For non-critical paths, it may choose lower drive strength and/or HVT cells to save power and area.



Key Takeaway:

A standard cell name tells you the function, drive strength and Vt type. Drive strength and Vt directly impact timing, power and area, and synthesis chooses the most suitable cells based on design constraints.



References:
 1. Synopsys Liberty User Guide
 2. Cadence Genus User Guide
 3. TSMC / Foundry Library Documentation
 4. Your shared PDF: "LIB file.pdf"

"Right Cell for the Right Purpose!"

Understanding the pin() Block in .lib File

Pin Level Information for Timing, Power and Electrical Constraints

Pins Define Behavior, Timing and Power!

1

What is a pin block ?

- The **pin block** defines information about each pin of a cell inside the .lib file.
- It specifies electrical, timing, power and functional details for the pin.
- Each pin (input, output, clock, etc.) has its **own** pin block inside the cell block.
- Pin information is used by synthesis, STA and place & route tools.

Example: pin block

```
pin (A) {
  direction : input ;
  capacitance : 0.0023 ;
  max_transition : 0.2 ;
  ...
}
```

2

What is direction ?

- direction** specifies the signal direction of the pin.
- It can be one of the following values:

Direction	Meaning
input	Pin receives signal into the cell.
output	Pin drives signal out of the cell.
inout	Pin can both receive and drive signal.
internal	Used for internal pins (not visible outside the cell).
clock	Specifies a clock pin (used for sequential cells).

Note: The direction tells the tool how the pin is used during timing, power and connectivity checks.

3

What is input pin and output pin ?

- Input pin** : Receives signal from the previous stage and drives the internal logic of the cell.
- Output pin** : Drives signal to the next stage and provides the output of the cell.
- Each input and output pin has its own electrical and timing characteristics (capacitance, transition, delay, etc.).

Example: Input and Output pins in an inverter (INV_X1)



Input pin example

```
pin (A) {
  direction : input ;
  capacitance : 0.0023 ;
  ...
}
```

Output pin example

```
pin (Y) {
  direction : output ;
  max_transition : 0.2 ;
  ...
}
```

4

What is clock pin & its capacitance ?

- Clock pin** is a special input pin used in sequential cells (like DFF, Latch, etc.).
- It is defined with **direction : clock**.
- Clock pin has an input capacitance, which represents the load seen by the clock network.
- This capacitance is used by CTS (Clock Tree Synthesis) for clock tree design and by STA for timing analysis.

Example: Clock pin in a D Flip-Flop

```
pin (CLK) {
  direction : clock ;
  capacitance : 0.0056 ; // clock input capacitance
  max_transition : 0.2 ;
  ...
}
```

Note: Clock pin capacitance is typically higher than normal input pins due to larger internal clock structures (e.g., clock buffers, gating logic).

5

What is rise and fall capacitance? What is max transition?

Rise and fall capacitance

- Capacitance can be different for rising and falling transitions.
- Some libraries specify separate values:

```
pin (A) {
  direction : input ;
  capacitance_rise : 0.0021 ;
  capacitance_fall : 0.0025 ;
  ...
}
```

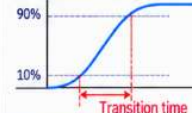
- rise capacitance** : load seen when signal transitions from 0 → 1.
- fall capacitance** : load seen when signal transitions from 1 → 0.
- If separate values are not specified, a single capacitance value is used for both.

Max transition (max slew)

- max_transition** specifies the maximum allowed input transition time (slew) at the pin.
- It is the time taken for the signal to transition from a low to high or high to low (typically 10% to 90%).
- If the transition at the pin exceeds this value, it violates the library constraint.

```
pin (A) {
  direction : input ;
  max_transition : 0.2 ; // in ns
  ...
}
```

Input Transition (Slew)



Transition time is measured between 10% and 90% of the signal amplitude.



Key Takeaway:

The **pin()** block defines the direction, capacitance, transition and other electrical/timing information for each pin. This information is critical for synthesis, STA, CTS and physical design.



References:
 1. Synopsys Liberty User Guide
 2. Cadence Genus / Innovus Documentation
 3. TSMC / Foundry Library Manual
 4. Your shared PDF: "LIB file.pdf"

"Correct Pin Information Enables Timing Closure!"

Input Capacitance and Output Load in .lib

Understanding their Impact on Delay, Transition and Timing Closure

Load Matters, Timing Gets Better!

1

What is input capacitance and why does an input have capacitance ?

- Input capacitance (C_{in})** is the equivalent capacitance seen at the input pin of a standard cell.
- It represents the total gate capacitance of the transistors connected to that input.
- An input has capacitance because the input is connected to the gates of MOS transistors, and a transistor gate behaves like a capacitor (due to gate oxide).
- This capacitance must be charged and discharged when the input signal switches, which consumes dynamic power and affects the delay of the driving cell.

Example: Input capacitance in .lib

```
pin (A) {
  direction : input ;
  capacitance : 0.0023 ; // in farads
  ...
}
```

Why does an input have capacitance ?



2

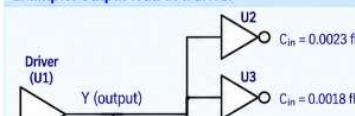
What is output load and how it affects delay and transition ?

- Output load (C_{load})** is the total capacitance driven by an output pin, which includes:
 - Input capacitances of all the fanout cells
 - Interconnect (wire) capacitance
 - Any additional load (e.g., test logic, buffers)
- A **larger output load** requires more current to charge and discharge, which results in:
 - Higher delay (slower switching)
 - Larger transition time (slew)

$$\text{Delay} \propto \frac{C_{\text{load}}}{\text{Drive Strength}}$$

$$\text{Transition} \propto \frac{C_{\text{load}}}{\text{Drive Strength}}$$

Example: Output load at a driver



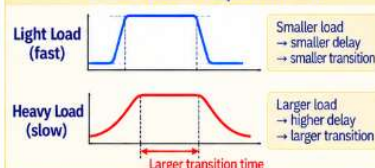
3

Why heavy load causes slow timing and what we have to do to fix it ?

Why heavy load causes slow timing ?

- A heavy load means a larger capacitance at the output.
- The driver has to charge and discharge more capacitance, which takes more time.
- This results in:
 - Higher propagation delay (cell delay)
 - Larger transition time (slew)
- If the transition becomes too large, it can violate the library **max_transition** constraint, leading to timing failures.

Effect of Load on Output Waveform



The input is connected to the gate of MOS transistors, and the gate oxide behaves like a capacitor.

C_{in} (gate capacitance)

Wire capacitance (e.g., 0.0010 fF)

$C_{in} = 0.0021$ fF

Total output load at U1.Y:
 $C_{load} = 0.0023 + 0.0018 + 0.0021 + 0.0010 = 0.0072$ fF

How to fix slow timing due to heavy load ?

- 1 Use a higher drive strength cell (e.g., X2, X4, X8).
- 2 Reduce fanout (buffer the net).
- 3 Optimize placement to reduce wire capacitance (shorter interconnect).
- 4 Use lower Vt cells (if power budget allows).
- 5 Re-architect the logic (e.g., break long nets, use repeaters).

Key Takeaway: Input capacitance and output load directly impact delay, transition, power and timing closure. Managing load through proper cell selection, buffering and placement is essential for meeting timing.

"Smaller Load, Faster Circuits, Better Designs!"

Standard Cells
Liberty File (.lib)
Power | Ground | Low Power

Power and Ground Pin in .lib File

Definition, Purpose and Importance in Timing, Power and Low Power Design

Power Defines the Possibilities of Silicon!

1 What is PG pin ?

- PG pin (Power/Ground pin) is a special pin in a standard cell used to supply power (VDD) and ground (VSS) to the transistors inside the cell.
- It does not carry functional signals like inputs or outputs.
- PG pins are usually defined inside the cell using the `pg_pin()` construct in a .lib file.
- A cell can have one or multiple power and ground pins (e.g., VDD, VSS, VDDH, VDDL, etc.).

Example: PG pin definition in .lib

```
pg_pin (VDD) {
    primary_power : true ;
    voltage_name : "VDD" ;
}

pg_pin (VSS) {
    primary_ground : true ;
    voltage_name : "VSS" ;
}
```

2 What is primary_power and primary_ground ?

- primary_power** : Indicates that this PG pin is the main power supply pin for the cell.
- primary_ground** : Indicates that this PG pin is the main ground return pin for the cell.
- A cell may have multiple power/ground pins, but only one of each is marked as primary.
- These attributes help EDA tools identify the main supply and ground pins for power analysis, cell placement and connectivity.

Example:

```
pg_pin (VDD) {
    primary_power : true ;
    voltage_name : "VDD" ;
}

pg_pin (VSS) {
    primary_ground : true ;
    voltage_name : "VSS" ;
}
```

Note: The names VDD and VSS are just common conventions. They can also be VCC/GND or any other names (e.g., VDDH, VDDL, VSS, VSSA, etc.).

3 What is voltage and why VDD and VSS defined in .lib ?

- Voltage** is the electrical potential difference between power and ground, which drives the current through the transistors.
- VDD** represents the positive supply voltage (e.g., 0.72 V, 1.0 V, 1.2 V, etc., depending on the technology).
- VSS** represents ground (0 V reference).
- VDD and VSS are defined in the .lib file so that EDA tools know the supply voltage for:
 - Power calculation (dynamic and leakage)
 - Timing characterization (delay, transition)
 - Electrical limits (min/max voltage)
 - Multi-Vt / multi-voltage cells

Voltage example (in .lib header)

```
library (my.lib) {
    voltage_unit : "1V" ;
    nominal_voltage : 0.72 ;
}
```

Typical values

Node	Nominal VDD
180 nm	1.8 V
65 nm	1.0 V
28 nm	0.9 V
7 nm	0.72 V

4 How does pg_pin help power analysis and low power design ?

In Power Analysis:

- PG pins identify the power and ground connections for each cell.
- Tools (e.g., PrimePower, Innovus) use this information to:
 - Compute dynamic and leakage power for each cell.
 - Build the power network (power rails and ground rails).
 - Accurately propagate supply voltage information.

In Low Power Design:

- Helps in multi-voltage (multi-supply) designs (e.g., VDDH, VDDL).
- Used to identify always-on (AON) cells vs. switchable power domains.
- Required for power gating (e.g., cells with separate VDD and VDDG).
- Enables proper isolation, level shifters and retention cells.
- Helps in accurate power intent (UPF) integration and verification.

Example: Standard cell with PG pins and signal pins

```
cell (NAND2_X1) {
    area : 0.526 ;
    pg_pin (VDD) {
        primary_power : true ;
        voltage_name : "VDD" ;
    }
    pg_pin (VSS) {
        primary_ground : true ;
        voltage_name : "VSS" ;
    }
    pin (A) { direction : input ; ... }
    pin (B) { direction : input ; ... }
    pin (Y) { direction : output ; ... }
}
```

Visual Representation

PG pins supply power to the transistors inside the cell, enabling its functionality.

Key Takeaway: PG pins (**primary_power** and **primary_ground**) define the main supply and ground connections of a cell. They are essential for accurate power analysis, timing, placement, and low power design (multi-voltage, power gating, UPF).

"No Power, No Performance!"

Standard Cells
Liberty File (.lib)
Synthesis | Logic Mapping

Cell Functionality – function Attribute in .lib

Defines the Logical Behavior of a Standard Cell

Logic in Words, Hardware in Silicon!

1 What is a Boolean function in .lib ?

- A Boolean function describes the logical relationship between a cell's inputs and outputs.
- It defines what the cell does, independent of timing, power or physical implementation.
- The function is written in Boolean expression using input pin names.
- It tells the tool the exact logic behavior of the cell.

Example: function of common cells

Cell	Logic Symbol	Boolean Function (in .lib)
INV		function : "!A" ;
AND2		function : "A & B" ;
OR2		function : "A B" ;
NAND2		function : "! (A & B)" ;
NOR2		function : "! (A B)" ;
XOR2		function : "A ^ B" ;

2 How does .lib describe cell functionality ?

- Inside the cell block, each output pin has a function attribute that defines its logic equation in terms of the input pins.
- The function is written using Boolean operators:
 - !(NOT) & (AND) | (OR) ^ (XOR)
- The expression uses the actual pin names defined in the cell (e.g., A, B, Y).

Example: NAND2_X1 in .lib

```
cell (NAND2_X1) {
    pin (A) { direction : input ; }
    pin (B) { direction : input ; }
    pin (Y) { direction : output ; }
    function : "! (A & B)" ;
}
```

3 How does synthesis use the function ?

- Synthesis reads the function to know the exact logic implemented by each standard cell.
- It uses this information to match the RTL logic with equivalent standard cells from the library.
- Timing, power and area are then considered to choose the best cell for the logic.
- Without the function, the tool cannot verify logical equivalence.

Example: RTL to cell mapping

RTL Logic	Synthesis (Genus)	Mapped Standard Cell
assign Y = ~(A & B); (Verilog RTL)	Reads function from .lib and finds matching cell	NAND2_X1 (from .lib)

4 How does Genus map RTL logic to a standard cell ?

- 1 **Parses RTL:** Understands the logic from Verilog/VHDL.
- 2 **Uses Boolean matching:** Compares RTL logic with the function of cells in the .lib file.
- 3 **Checks constraints:** Considers timing, power, area and design constraints (e.g., drive strength, Vt).
- 4 **Selects best cell:** Chooses the most suitable cell that matches the logic and meets the constraints.
- 5 **Creates netlist:** Replaces RTL logic with the selected standard cells.

Example Mapping in Genus

5 What happens if function information is incorrect ?

- The cell will behave differently than expected, leading to functional mismatch.
- Synthesis may map incorrect logic, causing wrong functionality in the netlist.
- This can result in simulation mismatches, silicon failure or non-functional design.
- Therefore, function information must be correct and verified by the library vendor.

Note: The function attribute defines the logical behavior of the cell. It is critical for correct synthesis, simulation and silicon functionality.

Key Takeaway: The **function** attribute in .lib defines the Boolean logic implemented by a cell. Synthesis tools like Genus use this information to map RTL logic to the correct standard cell, ensuring functional correctness.

"Correct Logic Builds Reliable Chips!"

Standard Cells
Liberty File (.lib)
Timing and Logic Behavior

Timing Arc and Unateness in .lib

Understanding Logical Influence and Timing Relationships

Know the Logic Influence for Better Timing Closure!

1 What is a Timing Arc ?

2 Positive Unate

3 Negative Unate

4 Non-Unate

- A **timing arc** represents the propagation delay from an input pin to an output pin.
- It defines how a transition at the input affects the output, including delay, transition and constraint information (setup/hold for sequential cells).
- Each arc in the .lib file has associated timing tables (e.g., cell_rise, cell_fall) indexed by input transition and output load.

Example: Timing arc in .lib (NAND2)

```
pin (A) {
  direction : input ;
  timing () {
    related_pin : "Y";
    timing_sense : negative_unate ;
    cell_rise ( ... ) { ... } // A↑→Y↑
    cell_fall ( ... ) { ... } // A↓→Y↓
  }
}
```

- A **positive unate** input means the output changes in the same direction as the input.
- If the input rises (0 → 1), the output also rises (or does not fall).
- If the input falls (1 → 0), the output also falls (or does not rise).
- Example cells:** Buffer (BUF), Non-inverting gate, AND (for each input), OR (for each input).

Example: Buffer (positive unate)

A (Input)	Y (Output)	Relation
0 → 1	0 → 1	Rises
1 → 0	1 → 0	Falls

timing_sense : positive_unate

Input and output move in the **same** direction.

- A **negative unate** input means the output changes in the opposite direction of the input.
- If the input rises (0 → 1), the output falls (or does not rise).
- If the input falls (1 → 0), the output rises (or does not fall).
- Example cells:** Inverter (INV), NAND (for each input), NOR (for each input).

Example: Inverter (negative unate)

A (Input)	Y (Output)	Relation
0 → 1	1 → 0	Falls
1 → 0	0 → 1	Rises

timing_sense : negative_unate

Input and output move in **opposite** directions.

- A **non-unate** input means the output does not have a consistent relationship with the input (it can both rise and fall depending on other inputs).
- The effect of the input on the output depends on the values of other inputs.
- Example cells:** XOR, XNOR, MUX, AOI/AOI gates (in some cases).

Example: XOR (non-unate)

A (B=0)	Y	A (B=1)	Y
0	0	0	1
1	1	1	0

timing_sense : non_unate

Input can cause both rise and fall at output depending on other inputs.

Key Takeaway: A **timing arc** defines the delay from an input to an output. **Unateness** tells whether the output moves in the same direction, opposite direction, or has no fixed relationship with the input. This information is critical for timing analysis and logic optimization.

"Understand Unateness, Design Better Circuits!"

Standard Cells | .lib File
Timing | Delay | STA

Cell Delay in .lib

Understanding Cell Delay, Rise/Fall Delay and Their Impact on Timing

Library Data Drives Timing Closure!

1 What is Cell Delay ?

- Cell delay is the propagation delay from an input transition to an output transition of a standard cell.
- It is the time taken for the output to respond after the input changes.
- The delay depends on the input transition time (slew), output load capacitance and the cell characteristics (from .lib).
- .lib provides cell delay values in timing tables for different input slews and output loads.

Example: Inverter delay

Cell delay is measured between the 50% point of input and 50% point of output transition.

2 What is cell_rise and cell_fall ?

- cell_rise** : Delay when the output makes a low → high transition (rising transition).
- cell_fall** : Delay when the output makes a high → low transition (falling transition).
- Both delays are provided in .lib as separate tables because the rising and falling delays are usually different.

Example: Timing tables in .lib (Inverter)

cell_rise (A → Y : low to high)

Input Slew (ns)	0	5	10	20
0.01	0.018	0.022	0.030	0.045
0.02	0.020	0.025	0.035	0.052
0.05	0.028	0.035	0.048	0.070

cell_fall (A → Y : high to low)

Input Slew (ns)	0	5	10	20
0.01	0.015	0.020	0.028	0.040
0.02	0.018	0.023	0.032	0.048
0.05	0.025	0.032	0.045	0.065

Note: Values are example numbers (not from a real library).

3 Why are rise and fall delays different ?

- Rise and fall delays are different because the internal pull-up (PMOS) and pull-down (NMOS) transistors have different strengths and characteristics.
- Typically, PMOS transistors are weaker (lower mobility) than NMOS transistors, so a low → high transition (rise) is slower than a high → low transition (fall) in many cells.
- Library characterizes both separately to capture the real silicon behavior.

Example: Inverter internal structure

Rise (low → high):

- Output is charged through PMOS
- PMOS is weaker → usually larger delay

Fall (high → low):

- Output is discharged through NMOS
- NMOS is stronger → usually smaller delay

Hence, rise and fall delays are different.

4 What affects cell delay and how does STA calculate delay from .lib ?

Factors that affect cell delay

- Input transition time (slew)
 - Slower input slew increases delay.
- Output load capacitance
 - Higher load increases delay.
- Cell drive strength
 - Higher drive strength (e.g., X4) gives lower delay.
- Cell internal structure
 - PMOS/NMOS sizing, threshold voltage (Vt).
- PVT conditions
 - Process, voltage and temperature.
- Signal direction (rise or fall)
 - Different arcs and unateness.

How STA calculates delay using .lib ?

- Determines the input transition time at the cell input.
- Determines the output load capacitance seen at the cell output (from net RC).
- Uses the .lib timing table (cell_rise or cell_fall) for the corresponding arc.
- Interpolates the delay value based on the input slew and output load (using bilinear interpolation).
- Adds the delay to the path and propagates the output transition to the next cell.

STA tools like PrimeTime read the .lib tables and use interpolation to get the exact delay for any slew and load value.

Interpolation example

Slew (ns)	Load (fF)	cell_rise (ns)
0.02	10	0.035
0.02	20	0.052

If actual load is 15 fF, STA interpolates between 0.035 and 0.052 to get the delay (e.g., ~0.044 ns).

Final delay used in STA:
Delay = f (input slew, output load, PVT, cell type, rise/fall arc)

Key Takeaway: Cell delay comes from .lib timing tables (cell_rise and cell_fall) and depends on input slew, output load, cell characteristics and PVT. STA interpolates these tables to get accurate delay values for timing analysis and closure.

"Accurate Libraries Enable Accurate Timing!"

Standard Cells
Liberty File (.lib)
Timing & Signal Integrity

Transition / Slew Tables in .lib

Understanding Output Transition and Its Dependence on Input Slew and Output Load

Clean Transitions Better Timing Reliable Designs!

1 What is Transition and Slew ?

- Transition (or Slew)** is the time taken for a signal to change from a low to high (rise) or high to low (fall) voltage.
- It is usually measured between 10% and 90% of the supply voltage (VDD).
- It indicates how fast the signal is changing.
- Transition is specified for both input pins (input slew) and output pins (output slew).

Rise and Fall Transition

Transition is measured from 10% to 90% of VDD

2 Rise and Fall Transition ?

- Rise transition:** Time taken for the output voltage to go from 10% to 90% of VDD (low → high).
- Fall transition:** Time taken for the output voltage to go from 90% to 10% of VDD (high → low).
- Rise and fall transitions can be different because PMOS and NMOS transistors have different strengths.

Example: Output waveform of an inverter

3 How input slew affects output slew ?

- If the input transition is slow (large slew), the output transition also becomes slower.
- A slow input means the transistor takes longer to switch, so the output node charges or discharges more slowly.
- If the input transition is fast (small slew), the output transition is faster.

Example: Output slew vs Input slew (for fixed load)

Input Slew (ns)	Output Slew (ns)	Rise	Fall
0.01	0.012	0.015	
0.05	0.020	0.026	
0.10	0.035	0.045	
0.20	0.060	0.085	

Observation: As input slew increases,

4 How output load affects transition ?

- A larger output load capacitance means more charge is required to change the output voltage.
- This takes more time, so the output transition (slew) becomes slower.
- A smaller load allows the output to transition faster.

Example: Output slew vs Output load (for fixed input slew)

Output Load (fF)	Output Slew (ns)	Rise	Fall
1	0.010	0.012	
5	0.018	0.022	
10	0.028	0.038	
20	0.050	0.070	
50	0.110	0.150	

Observation: As output load increases,

(industry standard).

output slew also increases.

Output values as output load increases, output slew becomes larger (slower).

5 Transition/Slew Tables in .lib

- The .lib file provides output transition tables (separate for rise and fall).
- These tables give the output slew for different combinations of input slew and output load.
- Tools (e.g., STA) use interpolation on these tables to get the exact output transition for a given input slew and load.

Example: cell_rise_transition table (from .lib)

input_slew (ns) ↓	output_load (fF) →				
	1	5	10	20	50
0.01	0.008	0.012	0.016	0.025	0.050
0.05	0.010	0.018	0.028	0.050	0.110
0.10	0.015	0.025	0.040	0.070	0.150
0.20	0.025	0.040	0.060	0.100	0.220

Example: cell_fall_transition table (from .lib)

input_slew (ns) ↓	output_load (fF) →				
	1	5	10	20	50
0.01	0.010	0.015	0.022	0.038	0.080
0.05	0.015	0.026	0.028	0.070	0.150
0.10	0.022	0.040	0.058	0.110	0.240
0.20	0.040	0.070	0.100	0.180	0.400

Key Takeaway:

Output transition (slew) depends on both input slew and output load. The .lib file provides rise and fall transition tables, and STA uses interpolation to find the exact output slew. Faster inputs and smaller loads result in faster output transitions.

"Understand Slew, Design for Speed!"

Lookup Tables in .lib

Standard Cells | .lib File
Timing Characterization
Lookup Tables (LUT)

Pre-characterized Data for Accurate Timing Analysis

From Library Data to Real Design Timing!

1 What is LUT in .lib and why is it required?

- A **Lookup Table (LUT)** in .lib is a pre-characterized table that stores timing or power values (e.g., cell delay, output transition, power) for different operating conditions.
- The values are obtained from detailed circuit simulations over a range of input slews and output loads.
- It is required because the cell delay and output transition are not constant; they vary with input transition (slew) and output load capacitance.
- Instead of using a complex analytical equation, EDA tools simply lookup (and interpolate) the value from the table, which is fast and accurate.

Why LUT is needed?

- Cell behavior is non-linear.
- Easy and fast lookup during STA.
- Captures real silicon behavior across PVT, slew and load.
- Supports accurate timing, power and signal integrity analysis.

2 What are index_1 and index_2?

- index_1** and **index_2** specify the two independent variables (axes) for the table.
- For timing tables (cell_rise, cell_fall, etc.):
 - index_1** → Input slew (transition time)
 - index_2** → Output load (capacitance)
- Each table entry gives the corresponding timing value (e.g., delay or output transition) for that combination of index_1 and index_2.

Example from .lib

```
cell_rise ("A -> Y") {
  index_1 ("0.01, 0.05, 0.1, 0.2"); // input slew (ns)
  index_2 ("1, 5, 10, 20"); // output load (fF)
  values (
    "0.008, 0.012, 0.016, 0.025",
    "0.010, 0.018, 0.028, 0.050",
    "0.015, 0.025, 0.040, 0.070",
    "0.025, 0.040, 0.060, 0.100" );
}
```

index_1 = input slew (rows), index_2 = output load (columns)

3 What are table values and input slew axis?

- Table values:** Each entry in the table is the characterized value (e.g., cell delay in ns or output transition in ns) for a specific input slew and output load combination.
- Input slew axis (index_1):** The row index of the table. It represents the different input transition times (slew) used during characterization. As we go down the rows, the input slew increases.

Example LUT Table (cell_rise)

		index_2: Output Load (fF) (columns)			
		1	5	10	20
index_1: Input Slew (ns) (rows)	0.01	0.008	0.012	0.016	0.025
	0.05	0.010	0.018	0.028	0.050
	0.10	0.015	0.025	0.040	0.070
	0.20	0.025	0.040	0.060	0.100

Table Values (e.g., cell delay in ns)

Visual Representation of LUT

How EDA Tool Uses LUT?

- Reads index_1, index_2 and table values from .lib
- For a given input slew and output load, it finds the corresponding table entry.
- If the exact value is not present, it performs interpolation (bilinear interpolation) between the neighboring points.
- The interpolated value is used as the cell delay or output transition in STA.

Example of Interpolation

- If input slew = 0.07 ns and output load = 7 fF:
- The tool finds the four nearest points from the table.
- Performs bilinear interpolation.
- Returns the estimated delay (e.g., ~0.032 ns).

This gives accurate timing without storing an infinite number of values.

Key Takeaway:

Lookup tables in .lib provide pre-characterized timing/power values indexed by input slew (index_1) and output load (index_2). EDA tools use interpolation to get accurate delay and transition values for any operating point.

"Characterized Data for Confident Timing!"

How STA Uses Look-Up Tables (LUTs)

Standard Cells | .lib File
STA | Lookup Tables | Interpolation

Mapping → Selecting → Reading → Interpolating → Getting Accurate Delay

Library Data into Real Timing Insights!

1 How does an STA tool map, select, and read delay values from a LUT?

- Map the cell**
 - STA reads the netlist and finds the cell type (e.g., NAND2_X1).
 - It opens the .lib file and locates the corresponding cell and timing arc (e.g., A → Y).
- Select the correct table**
 - For delay: uses cell_rise or cell_fall table (depending on output transition).
 - For output slew: uses rise_transition or fall_transition table.
- Get the design values**
 - Takes the input slew (from previous stage) and output load capacitance (from the net).
- Read/interpolate from LUT**
 - Finds the appropriate indices in the 2D table (index_1 = input slew, index_2 = output load).
 - Reads the exact value if present, otherwise interpolates (bilinear interpolation).
- Use the value**
 - The obtained delay (and output slew) is used for timing analysis (arrival time, slack, etc.).

Structure of a LUT in .lib (Example)

```
cell_rise ("A -> Y") {
  index_1 ("0.01, 0.05, 0.1, 0.2"); // input slew (ns)
  index_2 ("1, 5, 10, 20"); // output load (fF)
  values (
    "0.008, 0.012, 0.016, 0.025",
    "0.010, 0.018, 0.028, 0.050",
    "0.015, 0.025, 0.040, 0.070",
    "0.025, 0.040, 0.060, 0.100" );
}
```

Each value is the cell delay (in ns) for a combination of input slew (row) and output load (column).

Lookup Table (cell_rise)

Input Slew (ns) (index_1)	Output Load (fF) (index_2)			
	1	5	10	20
0.01	0.008	0.012	0.016	0.025
0.05	0.010	0.018	0.028	0.050
0.10	0.015	0.025	0.040	0.070
0.20	0.025	0.040	0.060	0.100

2 What happens when exact design values are missing, and how do you calculate them?

- In real designs, the input slew and output load will rarely match the exact values in the table.
- STA uses **bilinear interpolation** to calculate the value from the four nearest points.
- If the value is outside the table range, it may use extrapolation or clamp to the nearest boundary (tool dependent).

Example: Bilinear Interpolation

Given: Input slew = 0.07 ns, Output load = 7 fF

Slew	Load	
	5 (L1)	10 (L2)
0.05 (S1)	0.018	0.028
0.10 (S2)	0.025	0.040

Bilinear interpolation formula:

$$f(x, y) = (1 - t_x)(1 - t_y)f_{11} + t_x(1 - t_y)f_{12} + (1 - t_x)t_yf_{21} + t_xt_yf_{22}$$

where t_x and t_y are the normalized distances.

Result: Interpolated delay ≈ 0.032 ns (This value is between 0.018 and 0.040, as expected.)

3 How do input slew and output capacitance physically affect cell delay?

1 Effect of Input Slew (transition time)

- A larger input slew means a slower input transition (the signal takes more time to switch).
- The transistor stays longer in the threshold region, increasing the effective delay.
- Hence, as input slew increases, cell delay increases.

Bigger input slew → Larger delay

2 Effect of Output Load Capacitance

- A larger output load means more capacitance needs to be charged (for rising output) or discharged (for falling output).
- It takes more current (and time) to charge/discharge a larger capacitance.
- Hence, as output load increases, cell delay increases.

Bigger output load → Larger delay

Delay is roughly proportional to the RC time constant: $t_{delay} \approx R_{out} \times C_{load}$

Key Takeaway:

STA maps each cell and timing arc to the correct LUT in the .lib file, uses input slew and output load to read (or interpolate) delay values, and both larger input slew and output capacitance increase the cell delay due to slower signal transitions and higher RC charging/discharging time.

"Accurate Tables Enable Accurate Timing!"

Setup Time in .lib

Standard Cells | .lib File
Timing Checks | Sequential Cells

Definition, Tables, and Practical Interpretation

Meet Setup, Avoid Violations, Build Reliable Designs!

1 What is Setup Time and how is it defined in .lib ?

- Setup time** (t_{setup}) is the minimum time before the active clock edge that the data input must be stable, so that the flip-flop correctly captures the data.
- If data changes too close to the clock edge (less than setup time), the flip-flop may capture the wrong value (setup violation).
- In .lib, setup time is defined using "timing_type : setup_rising/falling" with a 2D lookup table indexed by data slew and clock slew.

.lib Syntax Example (setup_rising)

```
timing () {
  related_pin : "CK";           // clock pin
  constrained_pin : "D";         // data pin
  timing_type : setup_rising;
  rise_constraint ("setup_rising") {
    index_1 ("0.01, 0.05, 0.1, 0.2"); // data slew values
    index_2 ("0.01, 0.05, 0.1, 0.2"); // clock slew values
  }
}
```

2 What are setup_rising and setup_falling ?

- setup_rising**: Setup time requirement when the clock has a rising active edge ($\uparrow$). Data must be stable before the rising edge.
- setup_falling**: Setup time requirement when the clock has a falling active edge ($\downarrow$). Data must be stable before the falling edge.
- Both are provided because the required setup time can be different for rising and falling clock edges due to internal circuit implementation.

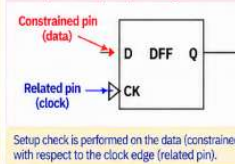
Example: Setup with clock edges



3 What are constrained pin and related pin ?

- constrained_pin**: The pin whose timing is being checked. For setup time, this is the data input pin (e.g., D).
- related_pin**: The pin that provides the reference event (clock). For setup time, this is the clock pin (e.g., CK).
- The setup timing arc is from the constrained pin (D) to the related pin (CK).
- Similarly, for hold time, the same pins are used, but with timing_type : hold_rising/hold_falling.

Example: D Flip-Flop Timing Arc



Setup check is performed on the data (constrained pin) with respect to the clock edge (related pin).

4 Why does setup time depend on data slew and clock slew ?

- Data slew**: Slower data transitions take longer to be recognized by the internal circuits, so the data needs to arrive earlier (higher setup time).
- Clock slew**: A slower clock edge makes the latching decision less sharp, increasing uncertainty, so setup time increases.
- Therefore, .lib provides a 2D table with setup time values for different combinations of data slew and clock slew.
- This allows STA tools to accurately model real cell behavior under different operating conditions.

Effect of Slew on Setup Time

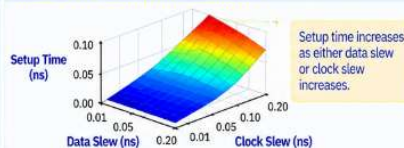
- Larger data slew $\rightarrow$ larger setup time
- Larger clock slew $\rightarrow$ larger setup time
- Faster transitions $\rightarrow$ smaller setup time

Setup time is not a constant value; it depends on both data slew and clock slew.

Example: setup_rising table from .lib (values in ns)

Data Slew (ns) (index_1) $\downarrow$	Clock Slew (ns) (index_2) $\rightarrow$			
	0.01	0.05	0.10	0.20
0.01	0.012	0.015	0.019	0.025
0.05	0.018	0.022	0.028	0.036
0.10	0.026	0.032	0.040	0.052
0.20	0.040	0.050	0.065	0.085

Graphical view of setup time dependency



Summary

Term	Meaning
Setup time	Minimum time before clock edge that data must be stable.
setup_rising	Setup time for rising clock edge.
setup_falling	Setup time for falling clock edge.
constrained_pin	Data pin (e.g., D).
related_pin	Clock pin (e.g., CK).
Depends on	Data slew and clock slew.



Key Takeaway:

Setup time in .lib is provided as a 2D lookup table (indexed by data slew and clock slew) with timing_type : setup_rising/setup_falling. It tells STA how early the data must arrive at the constrained pin with respect to the related clock edge for reliable operation.



"Right Timing Brings Correct Functionality!"



Standard Cells | .lib File
Timing Checks | Sequential Cells

Hold Time in .lib

Definition, Tables, and Impact on Timing

Prevent Hold Violations,
Build Robust Designs!

1 What is Hold Time and how is it defined in .lib ?

- Hold time** (t_{hold}) is the minimum time the data input must remain stable after the active clock edge, so that the flip-flop correctly captures the data.
- If the data changes too early (within the hold time window), the flip-flop may capture the wrong value (hold violation).
- In .lib, hold time is defined using "timing_type : hold_rising" or "timing_type : hold_falling" with a 2D lookup table indexed by data slew and clock slew.

.lib Syntax Example (hold_rising)

```
timing () {
  related_pin : "CK";           // clock pin
  constrained_pin : "D";         // data pin
  timing_type : hold_rising;
  rise_constraint ("hold_rising") {
    index_1 ("0.01, 0.05, 0.1, 0.2"); // data slew values
    index_2 ("0.01, 0.05, 0.1, 0.2"); // clock slew values
  }
}
```

2 What are hold_rising and hold_falling ?

- hold_rising**: Hold time requirement when the clock has a rising active edge ($\uparrow$). Data must remain stable after the rising edge.
- hold_falling**: Hold time requirement when the clock has a falling active edge ($\downarrow$). Data must remain stable after the falling edge.
- Both are provided because the hold time can be different for rising and falling edges due to internal circuit implementation (different paths and delays).

Example: Hold with clock edges



3 Why does hold depend on slew and how STA use hold constraints ?

- Hold time depends on data slew and clock slew because the internal delay through the flip-flop's input path (and clock path) varies with the transition times.
- A slow data slew can cause the data to change more gradually, effectively reducing the time it remains stable after the clock edge $\rightarrow$ larger hold time.
- A slow clock slew can move the internal latching point, also affecting the required hold time.
- STA uses the hold constraints from the .lib tables (indexed by data slew and clock slew) to determine the required hold time for each data-to-clock path. It then checks if the actual data arrival time is not earlier than the required hold time.

Larger data slew or clock slew $\rightarrow$ larger hold time requirement.

4 What causes a hold violation ?

- Data arrives too early at the flip-flop input after the clock edge (before the required hold time).
- This can happen due to:
 - Very short or zero logic path (e.g., direct connection or buffer only).
 - Fast data path (small cell delays).
 - Clock skew (capture clock arriving earlier than expected).
 - Useful skew that is negative for hold (e.g., launch clock later or capture clock earlier).
 - Process variations (faster silicon).
 - Improper clock tree synthesis or clock routing.

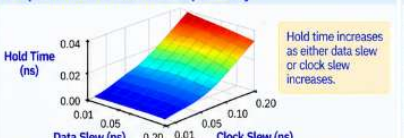
Example of Hold Violation



Example: hold_rising table from .lib (values in ns)

Data Slew (ns) (index_1) $\downarrow$	Clock Slew (ns) (index_2) $\rightarrow$			
	0.01	0.05	0.10	0.20
0.01	0.005	0.006	0.008	0.010
0.05	0.006	0.008	0.010	0.014
0.10	0.008	0.010	0.014	0.020
0.20	0.010	0.014	0.020	0.030

Graphical view of hold time dependency



Summary

Term	Meaning
Hold time	Minimum time data must remain stable after clock edge.
hold_rising	Hold time for rising clock edge.
hold_falling	Hold time for falling clock edge.
constrained_pin	Data pin (e.g., D).
related_pin	Clock pin (e.g., CK).
Depends on	Data slew and clock slew.



Key Takeaway:

Hold time in .lib is provided as a 2D lookup table (indexed by data slew and clock slew) with timing_type : hold_rising/hold_falling. STA checks that the data does not arrive earlier than the required hold time. Short paths, fast cells, or negative clock skew can cause hold violations.



"Fix Hold Issues before They Fail!"



Standard Cells | .lib File
Sequential Cells | Timing

Clock to Q Delay in .lib

Definition, Representation, and Impact on Setup Timing

Know Your Delays,
Meet Your Timing!

1 What is Clock to Q delay and how is it represented in .lib ?

- Clock to Q delay** (t_{cq}) is the time between the active clock edge and the output (Q) starting to change (becoming valid).
- It represents the internal delay of the flip-flop from the clock input (CK) to the data output (Q).
- In a .lib file, it is defined using a timing arc from the clock pin (related_pin) to the output pin (constrained_pin) with timing_type : clock_to_q and provided as 2D lookup tables (for rise and fall).
- There are separate tables for rising edge and falling edge, and for rising ($\uparrow$) and falling ($\downarrow$) transitions.

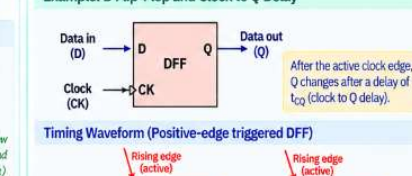
.lib Syntax Example (clock to Q)

```
timing () {
  related_pin : "CK";           // clock pin
  constrained_pin : "Q";         // output pin
  timing_type : clock_to_q;
  rise_transition ("clock_rising") {
    index_1 ("0.01, 0.05, 0.1, 0.2"); // clk slew
    index_2 ("1, 5, 10, 20");         // output load values
  }
}
```

2 What are rising_edge and falling_edge ?

- rising_edge**: The clock has a low-to-high transition ($\uparrow$). For a positive-edge triggered flip-flop, this is the active edge.
- falling_edge**: The clock has a high-to-low transition ($\downarrow$). For a negative-edge triggered flip-flop, this is the active edge.
- .lib provides clock-to-Q delay tables for both edges, e.g. clock_rising and clock_falling, because the delay can be different for each edge (even if only one edge is used in the design, both may be characterized).

Example: D Flip-Flop and Clock to Q Delay



3 Why is clock to Q delay important and how does it affect setup timing ?

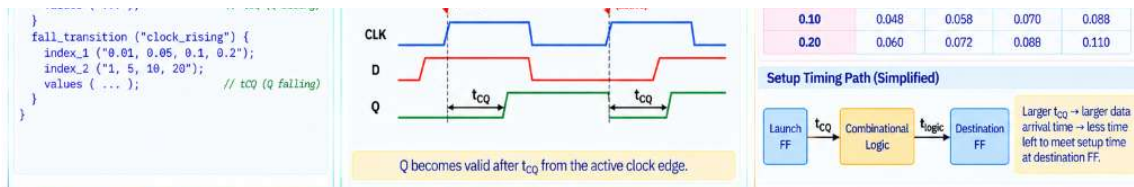
- Clock to Q delay represents the time it takes for the launching flip-flop to produce a valid output after the clock edge.
- In setup timing analysis, the data launched from the source flip-flop reaches the destination flip-flop after:

$$\text{Data arrival time} = t_{\text{cq}} + \text{combinational logic delay} + \text{interconnect delay}$$

- This data must arrive before the setup time (t_{setup}) of the destination flip-flop.
- Therefore, a larger t_{cq} increases the data arrival time and reduces the available time to meet setup, making it harder to satisfy the setup constraint.
- t_{cq} varies with clock slew (input transition at CK) and output load (capacitance at Q), so STA uses the .lib tables to get an accurate value.

Example: clock_to_q table (rising edge) from .lib (values in ns)

Clock Slew (ns) (index_1) $\downarrow$	Output Load (fF) (index_2) $\rightarrow$			
	1	5	10	20
0.01	0.035	0.042	0.050	0.062
0.05	0.040	0.048	0.058	0.072

**Key Takeaway:**

Clock to Q delay (t_{cq}) is the delay from the active clock edge to a valid output at Q, represented in .lib as a clock_to_q timing arc with separate tables for rising_edge and falling_edge. It directly affects the data arrival time and is critical for meeting setup timing.



"Accurate Delays
Enable Reliable Timing!"

Power Information inside .lib

Standard Cells | .lib File
Power Modeling | Low Power Design

From Library Data to Real Power Analysis

Understand Power
Design Efficient Chips!

**1 What power information is present in .lib?**

- The .lib file contains power models for each standard cell, in addition to timing and functional information.
- Power information is specified using the power and leakage_power groups.
- It includes tables for:
 - Internal (switching) power
 - Leakage power
 - Related power information such as when (conditions), related pins, and power units.

.lib Syntax Example (Power Section)

```

cell ("AND2_X1") {
  power () {
    internal_power () {
      related_pin : "A";
      when : "B";
      rise_power ("... table values ...");
      fall_power ("... table values ...");
    }
    leakage_power () {
      value : 1.2e-07; // leakage in W
    }
  }
}
  
```

2 What is internal power and switching power?

- Internal power** (also called **switching power**) is the power consumed inside the cell due to charging and discharging of internal capacitances when the cell switches.
- It depends on:
 - Input transition (slew)
 - Output load (capacitance)
 - Input activity (how often it switches)
 - The logic condition (when)

In .lib, internal power is modeled using 2D lookup tables (indexed by input slew and output load) for rise and fall power.

Example: Internal Power Table (rise_power)

Input Slew (ns)	1	5	10	20
0.01	1.2	2.8	4.1	6.5
0.05	1.8	4.2	6.0	9.1
0.10	2.6	6.0	8.5	12.8
0.20	4.1	9.5	13.2	19.0

Values are typically in nW or μ W (depends on library units)

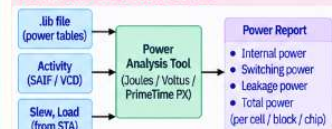
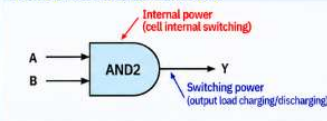
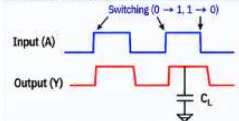
3 What is leakage power and dynamic power?

- Leakage power:**
 - Power consumed even when the cell is not switching (static power).
 - Caused by subthreshold leakage, gate leakage, junction leakage, etc.
 - In .lib, it is defined using the leakage_power group (usually a constant value, sometimes with voltage/temperature scaling).
 - Dominant in low-activity or standby mode designs.
- Dynamic power:**
 - Total power consumed due to switching activity.
 - It is the sum of internal power (cell internal switching) + switching power (output load charging/discharging).
 - Dynamic power depends on input activity, input slew, and output load.
 - In .lib, internal power is modeled explicitly, while the tool calculates switching power using the output capacitance and activity.

$$\text{Dynamic Power} = \text{Internal Power} + \text{Switching Power (Output Load)}$$

4 How do power tools use .lib?

- Power analysis tools (e.g., Cadence Joules, PrimeTime PX, Voltus) read the .lib file to get the power models for each cell.
- The tools use:
 - Activity information (from simulation, SAIF/VCD, or vectorless estimation)
 - Input slew and output load (from timing analysis)
 - Power tables (internal power)
 - Leakage values (from leakage_power section)
- They interpolate the table values (similar to timing) to find the power for the actual operating conditions.
- The total power is then reported at cell, block, and chip level (dynamic, leakage, and total).

Flow: How Power Tools Use .lib**Power Components in a Standard Cell****Power vs. Activity****Summary**

Term	Meaning
Internal power	Power consumed inside the cell due to internal switching (modeled in .lib using tables).
Switching power	Power to charge/discharge output capacitance.
Dynamic power	Internal power + switching power (activity dependent).
Leakage power	Static power when the cell is not switching (defined in leakage_power).
Power tools	Use .lib, activity, slew and load to calculate and report power at cell/block/chip level.

**Key Takeaway:**

The .lib file provides accurate power models (internal and leakage) for each cell. Power analysis tools use these models along with activity, slew, and load to compute dynamic, leakage, and total power, enabling reliable low-power design.



"Library Data
Powers a Smarter Tomorrow!"

Internal Power Tables in .lib

Standard Cells | .lib File
Power Modeling | Internal Power

Understanding Cell Internal Switching Power

Switching Inside the Cell
Consumes Power Too!

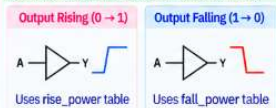
**1 What is Internal Power?**

- Internal power** is the power consumed inside a cell due to charging and discharging of the internal capacitances (internal nodes) when the cell switches its output.
- It does **not** include the power to charge external load (that is called switching power).
- It is provided in the .lib file using **internal_power** tables, indexed by input transition (slew) and output load.
- It is usually given separately for rising and falling output transitions.

Internal Power = Power consumed inside the cell during a switching event (0 → 1 or 1 → 0).

2 What are Rise and Fall Power?

- rise_power**: Internal power consumed when the cell output makes a low-to-high transition (0 → 1).
- fall_power**: Internal power consumed when the cell output makes a high-to-low transition (1 → 0).
- Both are provided as separate tables in the .lib file because the internal switching activity and internal nodes accessed can be different for rising and falling transitions.

Example: Output Transition and Internal Power**3 What affects Internal Power?**

- Input transition (slew)**
 - Slower input transitions increase internal power (more short-circuit current inside the cell).
 - Faster transitions reduce internal power.
- Output load**
 - Larger output load increases internal power because internal nodes may switch more and for longer time.
- Cell logic and design**
 - Complex cells (e.g., AOI/OAI, adders, muxes) have more internal nodes, hence higher internal power.
- Input data pattern / activity**
 - Different input combinations activate different internal nodes.
 - The .lib internal_power tables usually assume a default input pattern (often 50% activity on non-timing inputs).
- Supply voltage and PVT**
 - Higher VDD, lower Vt, or higher temperature increases internal power.

4 How does Switching Activity affect Internal Power?

- Switching activity (or toggle rate) determines how often the cell switches, so it directly scales the internal power.
- Internal power is usually reported per switching event (per transition).
- Average internal power is calculated as:

$$P_{\text{internal}} = \alpha \times P_{\text{event}}$$

where,

- α = switching activity (average transitions per cycle)
- P_{event} = internal power per transition (from .lib table, in nW or μ W)

Example:
If rise_power = 5 nW, fall_power = 4 nW and output toggles with 20% activity:
 $P_{\text{internal}} = 0.2 \times (5 + 4)/2 = 0.9 \text{ nW (approx.)}$

**Higher activity → higher internal power
Lower activity → lower internal power**

.lib Syntax Example (Internal Power)

```

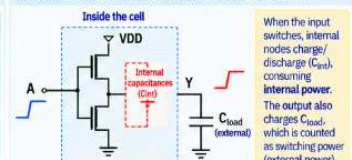
cell ("INV_X1") {
  power () {
    internal_power () {
      related_pin : "A";
      when : "A";
      rise_power ("... table values ..."); // input slew
      fall_power ("... table values ..."); // output load
    }
    leakage_power () {
      value : 1.2e-07; // leakage in W
    }
  }
}
  
```

Example: Internal Power Tables (Values in nW)

rise_power table					fall_p
Input Slew (ns)	1	5	10	20	Input (ns)
0.01	0.8	1.2	2.0	3.1	0.01
0.05	1.0	1.6	2.8	4.5	0.05
0.10	1.5	2.6	4.1	6.8	0.10
0.20	2.3	4.0	6.5	10.2	0.20

Power table				
New	Output Load (fF)			
	1	5	10	20
0.1	0.7	1.1	1.8	2.8
0.5	0.9	1.4	2.4	4.0
1.0	1.3	2.1	3.5	5.8
2.0	2.0	3.2	5.4	8.7

Values are in nW (can also be in μ W in some libraries). Higher slew and load → higher internal power.

Illustration: Where Does Internal Power Come From?**Key Takeaway:**

Internal power represents the power consumed inside the cell for each output transition. It depends on input slew, output load, cell design, and switching activity. Power tools read the .lib internal_power tables and multiply by the actual switching activity to estimate average internal power in the design.



"Understand Internal Switching.
Design for Lower Power."

Leakage Power and State Dependent Leakage

Standard Cells | .lib File
Power Modeling | Low Power Design

Understand Why It Happens, How It Is Modeled in .lib, and the Impact of Cell Threshold Voltage

No Switching
Still Consumes Power!

**1 What is Leakage Power and why it exists without switching?**

- Leakage power** is the static power consumed by a cell even when there is no switching activity.
- It exists due to subthreshold leakage, gate oxide

2 What is State Dependent Leakage?

- State dependent leakage means the leakage power of a cell depends on the input logic state (0 or 1) of its pins.
- Different input combinations turn ON/OFF different

3 Why LVT have higher leakage and HVT have less leakage?

- LVT** (Low Threshold Voltage) cells have a lower V_{th}, which makes transistors switch faster (higher performance), but increases subthreshold leakage exponentially.

leakage, and junction leakage in transistors.

- In modern CMOS (nm technologies), transistors are never completely OFF. Even when the gate is at 0 V, a small current flows from drain to source due to subthreshold conduction.
- This current flows continuously, so leakage power exists even when the circuit is idle (no switching).
- Leakage increases with higher temperature and lower threshold voltage (V_{th}).

Leakage Power = $V_{DD} \times I_{leakage}$

Why leakage exists? (Transistor in OFF state)

Small leakage current due to subthreshold conduction

Main leakage components:

- Subthreshold leakage → current flow even when $V_{GS} < V_{th}$
- Gate oxide leakage → tunneling through thin gate oxide
- Junction leakage → reverse biased p-n junction leakage

Lib Example: State Dependent Leakage

```
cell ("AND2_X1") {
  leakage_power () {
    when : "A & B"; // both inputs 1
    value : 2.5e-07; // leakage in Watts
  }
  leakage_power () {
    when : "A & !B"; // both inputs 0
    value : 8.0e-08;
  }
  leakage_power () {
    when : "!A & B"; // A=1, B=0
    value : 1.2e-07;
  }
  ...
}
```

Example: Leakage Power for Different Input States (AND2_X1)

A	B	Leakage Power (nW)
0	0	0.08
0	1	0.12
1	0	0.12
1	1	0.25

Leakage power changes with input state due to different internal transistor ON/OFF conditions.

- HVT** (High Threshold Voltage) cells have a higher V_{th} , which reduces subthreshold leakage (lower static power), but makes the cell slower.
- This is a fundamental trade-off between performance and leakage power.

Effect of Threshold Voltage on Leakage (Subthreshold Current)

$$I_{leakage} \propto e^{-\left(\frac{V_{th}}{n \times V_T}\right)}$$

As V_{th} increases, $I_{leakage}$ decreases exponentially.

Comparison of LVT, SVT and HVT Cells

Cell Type	Threshold Voltage (V_{th})	Leakage Power	Performance (Delay)	Use Case
LVT (Low V_{th})	Low (~0.2 - 0.3 V)	High	Fastest	Critical timing paths
SVT (Standard V_{th})	Nominal (~0.35 - 0.4 V)	Medium	Medium	General use
HVT (High V_{th})	High (~0.45 - 0.5 V)	Low	Slower	Non-critical paths, low power design

Leakage vs Threshold Voltage (Conceptual)

Higher V_{th} significantly reduces leakage but increases delay.

Key Takeaways:

- Leakage power exists due to physical device leakage mechanisms, even without switching.
- State dependent leakage is modeled in .lib using the "when" condition for different input states.
- LVT cells are faster but leak more, while HVT cells leak less but are slower → choose based on performance vs. power requirement.

"Control Leakage, Design Better Chips!"

Standard Cells | .lib File
Electrical Characteristics

Electrical Design Rules (EDR) in .lib

Define the electrical limits for reliable and high-quality design

Follow the Rules for a Robust Design!!

1 What electrical limits are defined in .lib?

- The .lib file specifies electrical design rules (limits) for each output pin of a cell.
- The main electrical limits are:
 - max_transition** (maximum allowed output slew)
 - max_capacitance** (maximum allowed load capacitance)
 - max_fanout** (maximum allowed fanout count)
- These limits guide synthesis and physical design tools to ensure signal integrity, timing accuracy, and reliable operation.

.lib Syntax Example (Output Pin Limits)

```
cell ("INV_X1") {
  pin ("Y") {
    direction : output;
    function : "!A";
    max_transition : 0.1; // in ns
    max_capacitance : 0.20; // in pF (or ff)
    max_fanout : 10;
  }
}
```

Values and units depend on the library (time_unit, capacitive_load_unit, etc.).

2 What is Max Transition?

- Max transition** is the maximum allowed output slew (rise/fall time) at the output pin of a cell.
- It limits how slowly or quickly the output voltage is allowed to change.
- If the output transition is worse (slower) than this limit, the cell violates the electrical rule, which can cause:
 - Incorrect timing (library data may not be valid)
 - Signal integrity issues
- Tools fix it by upsizing the cell or inserting buffers.

Transition (slew) = time to go from 10% → 90% of VDD (for rising) or 90% → 10% (for falling)

Output Transition (Slew)

Constraint: Output transition ≤ max_transition (from .lib)

3 What is Max Capacitance?

- Max capacitance** is the maximum allowed load capacitance that can be driven by the output pin of a cell.
- It represents the total capacitive load seen at the output (cell inputs + interconnect + other parasitics).
- If the load capacitance is higher than this limit, the cell may not be able to drive the load within valid timing and slew limits.
- Tools fix it by upsizing the cell, inserting buffers, or reducing the load.

Total Load Capacitance (C_L) = Cell input caps + Net (wire) cap + Other loads

Output Load Capacitance Example

Constraint: Total load capacitance ≤ max_capacitance (from .lib)

4 What is Max Fanout?

- Max fanout** is the maximum number of standard cell inputs that can be driven by the output pin.
- It limits the logical fanout count (not the actual capacitance).
- Even if the total capacitance is within limit, driving too many inputs can degrade signal quality and increase delay.
- Tools fix it by inserting buffers or using a higher drive strength cell.

Fanout Example

Constraint: Number of driven inputs ≤ max_fanout (from .lib)

Typical Values (Example - 7nm Library)

Drive Strength	max_transition (ns)	max_capacitance (pF)	max_fanout
X1	0.10	0.20	10
X2	0.09	0.40	20
X4	0.07	0.80	40
X8	0.05	1.60	80

Why These Limits Are Important?

- Ensure library characterization is valid.
- Maintain signal integrity and avoid waveform distortion.
- Help tools make correct timing and power optimizations.
- Prevent excessive delay and noise.
- Required for reliable silicon performance.

What Happens if These Limits Are Violated?

- Setup/hold timing may become inaccurate.
- Signal may not reach valid logic levels.
- Increased delay and transition time.
- Potential functional failure in silicon.
- Synthesis/PD tools report DRV (Design Rule Violation) and fix it by buffering, resizing, or re-structuring the logic.

Key Takeaway: .lib defines max_transition, max_capacitance, and max_fanout for each output pin. Following these electrical design rules (EDRs) ensures reliable timing, good signal integrity, and successful tapeout.

"Respect the .lib Limits, Build Better Chips!"

Standard Cells | .lib File
Design Rules | Physical Design

What Happens When DRVs Violate?

Impact, Examples and Fixes for a Robust Design

Fix DRVs Today for Better Silicon Tomorrow!

1 What is Max Transition Violation?

- Max transition** is the maximum allowed output slew (rise/fall time) at a cell output pin, defined in .lib (e.g., 0.1 ns).
- Violation occurs when the actual output slew is greater than the limit.
- Large transition makes the signal change slowly, causing more delay, higher power, and potential functional issues.

Ideal (within limit) vs **Violation (slew too large)**

Impact:

- Increased cell delay
- Higher dynamic power
- Possible signal integrity issues
- Can propagate to further timing violations

2 What is Max Capacitance Violation?

- Max capacitance** is the maximum allowed load capacitance at a cell output pin, defined in .lib (e.g., 0.20 pF).
- Violation occurs when the total load (sum of input caps of fanout cells + net capacitance) exceeds this limit.
- Excessive load means the cell has to charge/discharge a larger capacitance, increasing delay and output transition.

Total Load Capacitance (C_L) = $C_1 + C_2 + C_3 + C_{wire}$

Impact:

- Increased delay and output transition
- Higher dynamic power
- May cause transition violations
- Can reduce noise margin

3 What is Max Fanout Violation?

- Max fanout** is the maximum number of standard cell inputs that can be driven by a cell output, defined in .lib (e.g., 20).
- Violation occurs when the actual number of driven inputs exceeds this limit.
- Even if total capacitance is within limits, very high fanout can degrade signal quality, increase delay and transition, and burden the driver.

Fanout = 6 (Violation) Limit = 4

Impact:

- Increased delay and transition
- Higher power
- Signal integrity issues (more loading)
- May affect both timing and functionality

4 What Happens When DRVs Violate?

- Tools (Genus/Innovus) report Design Rule Violation (DRV) when any of the electrical limits (max_transition, max_capacitance, max_fanout) are exceeded.
- DRVs can cause timing failures, higher power, noise, and even functional issues in silicon.
- Physical design tools fix DRVs using buffering, cell upsizing, or restructuring the logic.

Example Tool Report (Innovus/Genus)

Type	Limit (from .lib)	Actual	Status
Transition	0.10 ns	0.25 ns	VIOLATED
Capacitance	0.20 pF	0.45 pF	VIOLATED
Fanout	20	35	VIOLATED

Overall Impact on Design:

- Timing may not meet constraints
- Higher power consumption
- Increased crosstalk and noise
- Potential functionality failure in worst cases
- Extra iterations in PD flow (increased effort and runtime)

5 What is Buffering?

- Buffering** means inserting an additional buffer cell in the net to reduce the load on the original driver.
- It splits a large load or high fanout into smaller loads, helping to meet max transition, max capacitance, and max fanout limits.

Before (High Load) vs **After Buffering**

Impact:

- Drive strength = 1 Falls DRV (High slow / load)
- Drive strength = 4 Lower slew, can drive larger load

6 What is Cell Upsizing?

- Cell upsizing means replacing the original cell with a higher drive strength version of the same cell (e.g., AND2_X1 → AND2_X4).
- A larger cell has stronger transistors, so it can drive more load, resulting in lower output transition and ability to meet DRVs.
- Upsizing increases area and power, so it is used only when needed.

Original Cell (AND2_X1) vs **Upsized Cell (AND2_X4)**

Impact:

- Increased delay and transition
- Higher power
- Signal integrity issues (more loading)
- May affect both timing and functionality

Comparison: Buffering vs Upsizing

Aspect	Buffering	Cell Upsizing
What it does	Inserts extra buffer stage	Replaces cell with higher drive strength
Effective for	High fanout, high load, transition	High load, transition
Area impact	Increases (extra cell)	Increases (larger cell)
Power impact	Slight to moderate (in extra cells)	Moderate (larger cell)
Delay impact	Adds one extra stage (but reduces slew overall)	Reduces delay
Common usage	Very common	Common

Other Fixing Techniques (Used by Tools)

- Insert buffers
- Upsize cells (use higher drive strength)
- Reduce fanout (rebalance logic)
- Resize gates in the logic path
- Re-optimize placement and routing (to reduce net capacitance)
- Logic restructuring (e.g., use tree structure)

(Violation)
(Within limit)
(Specs Limit)

Key Takeaway:

- DRVs (max. transition, max. capacitance, max. fanout) are defined in .lib and must be met for reliable design.
- Violations lead to higher delay, power, and potential functional issues.
- Tools fix DRVs using buffering, cell upsizing, or logic restructuring.

*"Meet the Rules Today.
Avoid the Failures Tomorrow!"*

PVT and Corners in .lib

Standard Cells | .lib File | PVT | Timing | Reliable Design

Understand Process, Voltage, Temperature Corners and Their Importance

Design for All Conditions,
Not Just Typical!

1 What is Process, Voltage and Temperature Corner?

- A **PVT corner** represents a specific combination of **Process (P)**, **Voltage (V)** and **Temperature (T)** conditions under which the standard cell library is characterized.

Process (P)

- Process corner models the variation in transistor manufacturing.
- Due to manufacturing variations, transistors can be:
 - Slow process (transistors are slower)
 - Typical process (nominal)
 - Fast process (transistors are faster)

Voltage (V)

- Voltage corner represents the supply voltage variation.
- Lower voltage → slower transistors (higher delay, lower power).
- Higher voltage → faster transistors (lower delay, higher power).

Temperature (T)

- Temperature corner represents the operating temperature variation.
- Higher temperature → slower transistors (higher leakage).
- Lower temperature → faster transistors (typically lower leakage).

2 Why are PVT Corners Required?

- Manufacturing, voltage and temperature vary in real silicon, so a single library at nominal condition is not enough.
- To ensure the design works correctly across all possible conditions (best-case and worst-case).
- Different corners are used for different analysis purposes:

Analysis	Corner Used	Reason
Setup timing	Slow corner (Slow P, Low V, High T)	Largest delay (worst-case)
Hold timing	Fast corner (Fast P, High V, Low T)	Smallest delay (best-case)
Power analysis (dynamic)	Typical corner	Represents average power
Leakage power	High temperature corner (e.g., TT/FF at high T)	Leakage is highest at high T

- Helps achieve a robust and reliable design.
- Used in MMC (Multi-Mode Multi-Corner) analysis with different .lib files for each corner.

3 What Does a .lib File Name Indicate?

- The library file name usually encodes the process, voltage, temperature and sometimes the threshold flavor (LVT, SVT, HVT).
- By looking at the file name, we can quickly identify the characterization conditions of that library.

Example File Name:

tsmc7nm_tt_0p72v_25c.lib

Process (7nm)
Corner (TT)
Voltage (0.72 V)
Temperature (25 °C)

Token	Meaning / Possible Values
Process node	7nm, 5nm, 28nm, etc.
Process corner	ss (slow), tt (typical), ff (fast)
Voltage	0p72v (0.72 V), 0p8v (0.8 V), 1p0v (1.0 V), etc.
Temperature	-40c, 25c, 125c, etc.
Threshold flavor (optional)	lvt (low V_{th}), svt (standard V_{th}), hvt (high V_{th})
File extension	.lib (Liberty file)

PVT Combination Example

Corner Name	Process (P)	Voltage (V)	Temperature (T)	Usage
SS	Slow	Low (e.g., 0.72 V)	High (125 °C)	Setup (worst-case)
TT	Typical	Nominal (e.g., 0.8 V)	25 °C	Power (typical)
FF	Fast	High (e.g., 0.88 V)	Low (-40 °C)	Hold (best-case)

Effect of PVT on Cell Delay (Conceptual)

Condition	Relative Delay	Explanation
Slow P, Low V, High T	Highest	Transistors are slowest
Typical	Medium	Nominal condition
Fast P, High V, Low T	Lowest	Transistors are fastest

More Example .lib File Names

File Name	Indicates
tsmc7nm_ss_0p72v_125c.lib	7nm, Slow, 0.72 V, 125 °C
tsmc7nm_tt_0p8v_25c.lib	7nm, Typical, 0.8 V, 25 °C
tsmc7nm_ff_0p88v_-40c.lib	7nm, Fast, 0.88 V, -40 °C
tsmc7nm_tt_0p72v_25c_hvt.lib	7nm, Typical, 0.72 V, 25 °C, High V_{th}
tsmc5nm_ss_0p75v_125c_lvt.lib	5nm, Slow, 0.75 V, 125 °C, Low V_{th}

Key Takeaways:

- PVT corners model real-world variations in process, voltage and temperature.
- Multiple .lib files are required to ensure correct timing, power and functionality across all conditions.
- The library file name gives a quick indication of the process node, corner (ss/tt/ff), voltage and temperature (and sometimes Vth flavor).

*"Different Corners,
Same Reliable Design!"*

(Violation)
(Within limit)
(Specs Limit)

PVT and Corners in .lib

Standard Cells | .lib File | PVT | Timing | Reliable Design

Understand Process, Voltage, Temperature Corners and Their Importance

Design for All Conditions,
Not Just Typical!

1 What is Process, Voltage and Temperature Corner?

- A **PVT corner** represents a specific combination of **Process (P)**, **Voltage (V)** and **Temperature (T)** conditions under which the standard cell library is characterized.

Process (P)

- Process corner models the variation in transistor manufacturing.
- Due to manufacturing variations, transistors can be:
 - Slow process (transistors are slower)
 - Typical process (nominal)
 - Fast process (transistors are faster)

Voltage (V)

- Voltage corner represents the supply voltage variation.
- Lower voltage → slower transistors (higher delay, lower power).
- Higher voltage → faster transistors (lower delay, higher power).

Temperature (T)

- Temperature corner represents the operating temperature variation.
- Higher temperature → slower transistors (higher leakage).
- Lower temperature → faster transistors (typically lower leakage).

2 Why are PVT Corners Required?

- Manufacturing, voltage and temperature vary in real silicon, so a single library at nominal condition is not enough.
- To ensure the design works correctly across all possible conditions (best-case and worst-case).
- Different corners are used for different analysis purposes:

Analysis	Corner Used	Reason
Setup timing	Slow corner (Slow P, Low V, High T)	Largest delay (worst-case)
Hold timing	Fast corner (Fast P, High V, Low T)	Smallest delay (best-case)
Power analysis (dynamic)	Typical corner	Represents average power
Leakage power	High temperature corner (e.g., TT/FF at high T)	Leakage is highest at high T

- Helps achieve a robust and reliable design.
- Used in MMC (Multi-Mode Multi-Corner) analysis with different .lib files for each corner.

3 What Does a .lib File Name Indicate?

- The library file name usually encodes the process, voltage, temperature and sometimes the threshold flavor (LVT, SVT, HVT).
- By looking at the file name, we can quickly identify the characterization conditions of that library.

Example File Name:

tsmc7nm_tt_0p72v_25c.lib

Process (7nm)
Corner (TT)
Voltage (0.72 V)
Temperature (25 °C)

Token	Meaning / Possible Values
Process node	7nm, 5nm, 28nm, etc.
Process corner	ss (slow), tt (typical), ff (fast)
Voltage	0p72v (0.72 V), 0p8v (0.8 V), 1p0v (1.0 V), etc.
Temperature	-40c, 25c, 125c, etc.
Threshold flavor (optional)	lvt (low V_{th}), svt (standard V_{th}), hvt (high V_{th})
File extension	.lib (Liberty file)

PVT Combination Example

Corner Name	Process (P)	Voltage (V)	Temperature (T)	Usage
SS	Slow	Low (e.g., 0.72 V)	High (125 °C)	Setup (worst-case)
TT	Typical	Nominal (e.g., 0.8 V)	25 °C	Power (typical)
FF	Fast	High (e.g., 0.88 V)	Low (-40 °C)	Hold (best-case)

Effect of PVT on Cell Delay (Conceptual)

Condition	Relative Delay	Explanation
Slow P, Low V, High T	Highest	Transistors are slowest
Typical	Medium	Nominal condition
Fast P, High V, Low T	Lowest	Transistors are fastest

More Example .lib File Names

File Name	Indicates
tsmc7nm_ss_0p72v_125c.lib	7nm, Slow, 0.72 V, 125 °C
tsmc7nm_tt_0p8v_25c.lib	7nm, Typical, 0.8 V, 25 °C
tsmc7nm_ff_0p88v_-40c.lib	7nm, Fast, 0.88 V, -40 °C
tsmc7nm_tt_0p72v_25c_hvt.lib	7nm, Typical, 0.72 V, 25 °C, High V_{th}
tsmc5nm_ss_0p75v_125c_lvt.lib	5nm, Slow, 0.75 V, 125 °C, Low V_{th}

Key Takeaways:

- PVT corners model real-world variations in process, voltage and temperature.
- Multiple .lib files are required to ensure correct timing, power and functionality across all conditions.
- The library file name gives a quick indication of the process node, corner (ss/tt/ff), voltage and temperature (and sometimes Vth flavor).

*"Different Corners,
Same Reliable Design!"*

(Violation)
(Within limit)
(Specs Limit)

Slow, Typical and Fast Corners

Standard Cells | .lib File | PVT | Timing | Signoff

Understand Their Meaning, Usage and Impact on Timing and Power

Different Corners
for Different Checks!

1 What are Slow, Typical and Fast Corners?

- A **PVT corner** is a specific combination of **Process (P)**, **Voltage (V)** and **Temperature (T)** under which the standard cell library is characterized.
- Process variation causes transistors to be faster or slower than nominal.
- Based on these variations, we get three main corners: Slow, Typical and Fast.

Corner	Process	Voltage	Temperature	Transistor Speed	Cell Delay
Slow (SS)	Slow (process - larger delay)	Low V	High T	Slowest	Highest delay
Typical (TT)	Nominal	Nominal	Nominal	Nominal	Medium delay
Fast (FF)	Fast (process - smaller delay)	High V	Low T	Fastest	Lowest delay

Note: Actual values (V, T) depend on technology.
Example for 7nm:

2 Why is Slow Corner Used for Setup?

- Setup check ensures the data reaches before the clock edge.
- In the slow corner (SS): Slow process, Low V, High T, transistors are slow → cell delays are maximum.
- This represents the **worst-case (longest) delay path**.
- If the design meets setup at this worst case, it will meet setup at all other (faster) conditions.

Slow corner = Largest delay
→ Worst case for setup

Setup Check (Slow Corner)

3 Why is Fast Corner Used for Hold?

- Hold check ensures the data does not arrive too early (after the clock edge).
- In the fast corner (FF): Fast process, High V, Low T, transistors are fast → cell delays are minimum.
- This represents the **worst-case (shortest) delay path**.
- If the design meets hold at this best-case (minimum delay), it will meet hold at all other (slower) conditions.

Fast corner = Smallest delay
→ Worst case for hold

Hold Check (Fast Corner)

SS : 0.72 V, 125 °C | TT : 0.80 V, 25 °C | FF : 0.88 V, -40 °C

4 Which Corner is Used for Leakage ?

- Leakage power is highest at high temperature.
- Process type (especially low V_{th} transistors) also affects leakage.
- Therefore, leakage analysis is usually done at:
 - High temperature corner (e.g., SS at 125 °C or TT/FF at high T)**
- Foundries specify the recommended corner for leakage in the library documentation (often worst-case high T corner).

Summary: Which Corner is Used for What ?

Analysis Type	Corner Used	Reason
Setup Timing	Slow (SS)	Largest delay (worst-case)
Hold Timing	Fast (FF)	Smallest delay (worst-case)
Functional Timing (at nominal)	Typical (TT)	Nominal condition
Leakage Power	High temperature corner (e.g., SS at high T)	Leakage is highest at high temperature
Dynamic Power	Typically TT (or multiple)	Represents average activity (also checks at other corners)

Impact of PVT on Cell Delay and Leakage

Parameter	Increase	Effect on Delay	Effect on Leakage
Process (slower)	Larger transistors (lower drive)	Increase	Increase
Process (faster)	Smaller transistors (higher drive)	Decrease	Decrease
Voltage (V)	Lower V	Increase	Decrease
	Higher V	Decrease	Increase
Temperature (T)	Higher T	Increase	Increase
	Lower T	Decrease	Decrease

Key Takeaways:

- Slow corner (SS) → worst case for setup (largest delay).
- Fast corner (FF) → worst case for hold (smallest delay).
- Leakage analysis → use high temperature corner (usually SS at high T).
- Using the correct corners ensures a robust design across all real silicon variations.

"Check the Right Corner, Close Timing with Confidence!"

Process, Voltage and Temperature Effects

How PVT Variations Affect Cell Timing and Why Multiple Corners Are Required

Same Design, Different Conditions, Different Delays!

1 How does PVT affect timing ?

1. Process (P)

- Process variation changes the transistor threshold voltage (V_{th}), channel length and mobility.
- Slow process (SS) → transistors are slower (lower drive current) → higher delay.
- Fast process (FF) → transistors are faster (higher drive current) → lower delay.

Slow Process (SS)
Larger V_{th}
(Lower drive current)

Fast Process (FF)
Smaller V_{th}
(Higher drive current)

2. Voltage (V)

- Lower supply voltage (V_{DD}) reduces drive current → higher delay.
- Higher supply voltage increases drive current → lower delay.
- Delay roughly varies as:

$$\text{Delay} \propto \frac{1}{(V_{DD} - V_{th})^\alpha} \quad (\alpha \approx 1 \text{ to } 2)$$

Effect of Voltage on Delay

Lower V_{DD} → Higher delay

3. Temperature (T)

- Higher temperature reduces carrier mobility → higher delay (for most nodes).
- Lower temperature increases mobility → lower delay.
- At advanced nodes ($< 7\text{nm}$), temperature inversion may occur (leakage increases with lower T due to quantum effects).

Effect of Temperature on Delay

Higher T → Higher delay

2 Which PVT combination is slow and which one is fast ?

Worst-Case Slow Corner (Largest Delay)

- Process : Slow (SS)
- Voltage : Low (V_{min})
- Temperature : High (T_{max})
- Notation : SS / V_{min} / T_{max}
- Used for : Setup timing (worst-case delay)

Best-Case Fast Corner (Smallest Delay)

- Process : Fast (FF)
- Voltage : High (V_{max})
- Temperature : Low (T_{min})
- Notation : FF / V_{max} / T_{min}
- Used for : Hold timing (best-case delay)

Typical (Nominal) Corner

- Process : Typical (TT)
- Voltage : Nominal (V_{nom})
- Temperature : Nominal (T_{nom})
- Notation : TT / V_{nom} / T_{nom}
- Used for : Power analysis (dynamic power), functional checks, early design stages

3 Why must PD check multiple PVT corners ?

- Manufacturing variations: Each chip can be faster or slower due to process variation.
- Operating conditions: Voltage and temperature change in real use (e.g., low power mode, high temperature environment).
- Different corners stress different aspects:
 - Slow corner → worst-case delay (setup)
 - Fast corner → best-case delay (hold)
 - High temperature → worst-case leakage
 - Low temperature → possible leakage increase (advanced nodes)
- Ensures the design meets timing, power, and functionality under all conditions (MIMC – Multi-Mode Multi-Corner).
- Avoids silicon failure in real-world scenarios (across PVT, modes, voltages, and RC corners).

A design that works at one corner may fail at another corner if multiple PVT corners are not checked.

Common PVT Corners and Their Impact

Corner	Process (P)	Voltage (V)	Temperature (T)	Relative Delay	Main Use Case
SS	Slow	Low (V_{min})	High (T_{max})	Largest (slowest)	Setup timing
TT	Typical	Nominal	Nominal	Medium	Power analysis, functional
FF	Fast	High (V_{max})	Low (T_{min})	Smallest (fastest)	Hold timing
SF	Slow	High	Low	Medium	On-chip variation
FS	Fast	Low	High	Medium	On-chip variation

Key Takeaways:

- PVT affects timing because it changes transistor drive current and switching speed.
- Slow corner (SS, V_{min} , T_{max}) gives the largest delay → used for setup.
- Fast corner (FF, V_{max} , T_{min}) gives the smallest delay → used for hold.
- Multiple PVT corners must be checked to ensure a robust design across all real-world conditions.

"Design for All Corners, Build for the Real World!"

Liberty Timing Models

NLDM vs CCS vs ECSM – Understand the Models, Usage and Accuracy

Same Cell, Different Models, Different Accuracy!

1 What is NLDM, CCS and ECSM ?

NLDM (Non-Linear Delay Model)

- Traditional liberty timing model.
- Uses 2D lookup tables (LUTs) for cell delay and output transition.
- Characterized with a simple pin-to-pin delay and output slew model.
- Assumes linear ramp at input and output.
- Widely used for synthesis and early design stages.

Modeling style: Voltage-based, table lookup

CCS (Composite Current Source)

- Advanced timing model.
- Models the cell as a time-varying current source (instead of simple delay tables).
- Captures non-linear waveforms, output shape, and current behavior.
- More accurate for advanced technology nodes (sub-7nm and below).
- Mandatory for signoff (PrimeTime/Tempus).

Modeling style: Current-based, time-varying waveform

ECSM (Effective Current Source Model)

- A simplified current source model.
- Uses an effective current source to model driver behavior.
- More accurate than NLDM but less detailed than CCS.
- Good trade-off between accuracy and runtime.
- Historically used in some flows (now CCS is preferred).

Modeling style: Effective current-based

2 Why Different Timing Models Exist ?

- At advanced nodes, simple delay tables (NLDM) are not sufficient because:
 - Waveforms are non-linear (not simple linear ramps).
 - Interconnect parasitics (R, C) are highly resistive and capacitive.
 - Receiver effect (Miller capacitance) and waveform distortion become significant.
 - Need higher accuracy for signoff timing, noise, and SI analysis.
- Different models provide a trade-off between:

Accuracy	Runtime	Complexity
NLDM	Low	Low
ECSM	Medium	Medium
CCS	High	High

3 Which Model is More Accurate and Used in Synthesis ?

- Accuracy (from low to high):** NLDM < ECSM < CCS
- Used in Synthesis:** NLDM (most common)
 - Provides good enough accuracy with fast runtime.
 - Captures basic cell delay and transition.
 - Widely supported and sufficient for early optimization.
- Used for Signoff (PrimeTime/Tempus):** CCS (mandatory at advanced nodes)
 - Provides highest accuracy with real waveform modeling.
 - Required for signoff timing and SI analysis.
 - ECSM may be used in some intermediate flows, but CCS is preferred today.

4 Waveform Modeling Comparison

NLDM (Voltage-based)

Input (linear ramp) → Output (linear ramp)

Assumes linear ramp waveforms

ECSM (Effective Current Source)

Input (realistic) → Output (more realistic)

Uses effective current source, better waveform shape

CCS (Composite Current Source)

Input (real waveform) → Output (real waveform)

Models actual current waveform (highest accuracy)

Comparison Table

Feature	NLDM	ECSM	CCS
Modeling approach	Voltage-based (LUT)	Effective current source	Time-varying current source
Accuracy	Low	Medium	High (highest)
Waveform modeling	Linear ramp	Improved (effective)	Real, non-linear waveform
Captures SI effects	No	Partial	Yes
Runtime	Fast	Moderate	Slower
Typical usage	Synthesis, early STA	Some flows (intermediate)	Signoff STA (PrimeTime/Tempus)
Preferred at advanced nodes	No	Not preferred (legacy)	Yes (mandatory)

Example from .lib File

NLDM style

```
cell (AND2_X1) {
  pin (Y) {
    timing {
      rail_rise (table 30) {
        index 1 (0.01, 0.02, 0.05);
        index 2 (0.10, 0.02, 0.05);
        values ("");
      }
    }
  }
}
```

ECSM style

```
cell (AND2_X1) {
  pin (Y) {
    timing {
      rail_rise {
        rail_rise {
          // effective current data
          rise_waveform ("");
          fall_waveform ("");
        }
      }
    }
  }
}
```

CCS style

```
cell (AND2_X1) {
  pin (Y) {
    timing {
      rail_rise {
        rail_rise {
          // time-varying current
          rise_waveform ("");
          fall_waveform ("");
        }
      }
    }
  }
}
```

Key Takeaways:

- NLDM, ECSM and CCS are different liberty timing models with increasing accuracy.
- Different models exist to capture real silicon behavior and advanced node effects.
- NLDM is commonly used in synthesis, while CCS is most accurate and used for signoff.
- At advanced nodes, CCS is the industry standard for reliable timing closure.

"Better Models Lead to Better Silicon!"

.lib in the ASIC Flow

Same Library, Different Uses at Different Stages

.lib Powers Every Stage of ASIC Design!

RTL (.v) → Synthesis (Genus) → Floorplan & Placement (Innovus) → Clock Tree Synthesis (Innovus) → Routing (Innovus) → Signoff (STA / Tempus) → GDSII (Tape-out)

.lib is used in multiple stages to provide timing, power, area and electrical information about standard cells

1 How Synthesis Uses .lib ?

(Genus / DC)

Role of .lib in Synthesis

- Provides timing (cell delay, transition) for technology mapping.
- Provides area information to choose optimal cells.
- Provides resource information (forward, backward, backward)

2 How PnR Uses .lib ?

(Innovus / ICC2)

Role of .lib in PnR

- Uses cell area (from .lib) for placement and utilization.
- Uses timing data (delay, transition) for timing-aware placement.
- Uses electrical limits (max transition, max capacitance).

3 How STA Uses .lib ?

(PrimeTime / Tempus)

Role of .lib in STA

- Provides accurate timing data for all standard cells (cell delay, transition, setup, hold, clock-to-Q).
- Provides timing arc information (combinational and sequential).

Provides power information (internal, switching, leakage) for low-power optimization.

- Provides electrical limits (max_transition, max_capacitance, max_fanout) for initial DRV fixing.
- Provides cell functionality (boolean logic, timing arcs).
- Uses wire load model (defined in .lib) to estimate interconnect delay (before placement).

How It Works ?

- 1 Reads .lib and builds a technology database.
- 2 Maps RTL to standard cells using timing, area and power data.
- 3 Evaluates multiple cell options (e.g., AND2_X1 vs AND2_X4) using delay and area tables.
- 4 Performs optimizations (timing, area, power, DRV).
- 5 Generates synthesized netlist (.v) with standard cells from the library.

Example: Cell Selection in Synthesis

```
RTL: Y = A & B
```

Genus checks .lib timing/area/power

Chooses best cell:
AND2_X1 (slower, smaller area)
AND2_X4 (faster, larger area)
AND2_X8 (even faster)

Usage at Different PnR Stages

Stage	How .lib is Used	Key Information from .lib
Floorplanning	Estimate power, place large macros	Cell area, power
Placement	Timing-aware placement, legalization	Area, delay, transition, max_cap, max_tran
CTS	Select and size clock cells	Clock cell delay, transition, drive strength
Routing	Estimate RC and fix DRVs	Max_cap, max_tran, max_fanout
Post-Route Optimization	Fix timing and DRVs	Delay, transition, power, electrical limits

Example: Timing-Aware Placement

.lib (cell delay, area, max_tran/cap) → Innovus (Placement) → Cells placed to meet timing and electrical constraints

STA Flow with .lib

- 1 Read .lib (for all required PVT corners).
- 2 Read netlist (.v) and extracted parasitics (.spef).
- 3 Build timing graph using .lib timing arcs.
- 4 Calculate path delays, setup and hold slack.
- 5 Check transition, capacitance and other constraints.
- 6 Perform MMMC analysis using multiple .lib files.

Example: Timing Calculation

Launch FF (CK) → Delay from .lib (cell_rise/fall) → Delay from .lib (table lookup) → Delay from .lib → Setup/Hold from .lib (FF timing) → Capture FF (CK)

Key Takeaways:

- .lib is the backbone of the ASIC flow, used from synthesis to signoff.
- Each stage uses different information from .lib (timing, power, area, electrical limits).
- Accurate .lib (with correct PVT corners) is essential for a reliable and silicon-proven design.

"Same Library, Many Uses, One Successful Chip!"

Standard Cells | .lib File
Verify Before You Use

First Check After Receiving .lib
What Should a PD Engineer Check First ?

Check Early, Avoid Problems Later !

- 1 Check File Availability**
 - Ensure the .lib file is complete and not corrupted.
 - Check file size (should be reasonable, not 0 bytes).
 - Verify you have all required corners (e.g., SS, TT, FF).
 - Check if the file opens properly.

Useful Linux commands

```
ls -lh *.lib # Check file size
head -20 mylib.lib # View header
grep "Library" mylib.lib # Check library name
```
- 2 Check PVT Corner**
 - Identify the process, voltage and temperature conditions of the .lib file.
 - Ensure you have all required corners (e.g., SS, TT, FF).
 - Match the corners with project requirements.

Example from .lib:

```
library (tsmc7nm_tt_0p72v_25c) {
  operating_conditions (ss_0p72v_25c) {
    process : 1.0 ; # slow process
    voltage : 0.72 ;
    temperature : 25.0 ;
  }
}
```
- 3 Check Units**
 - Verify time, voltage, current and capacitance units.
 - Wrong units can lead to completely incorrect timing and power analysis.

Example from .lib:

```
time_unit : "ips" ;
voltage_unit : "1V" ;
current_unit : "1uA" ;
capacitive_load_unit (1, ff) ;
power_unit : "1mW" ;
```

Make sure these units match the tool setup and other libraries (.lib, .db, .lef).
- 4 Check Technology**
 - Verify the technology node (e.g., 7nm, 5nm).
 - Ensure it matches the project technology.

Library: tsmc7nm_tt_0p72v_25c
Technology: 7nm FinFET
Corner: Typical Process, 0.72V, 25C
Generated by: Foundry

Using the wrong technology .lib can cause mismatch in timing, power and physical design.
- 5 Check Library Name**
 - Verify the library name and naming convention.
 - Check if the name clearly indicates PVT, Vth, etc.
 - Ensure it matches with other files (.db, .lef).

Example:

```
library (tsmc7nm_tt_0p72v_25c) {
  ...
}
```
- 6 Check Required Cells**
 - Ensure all basic cells required for synthesis/PNR are present (e.g., INV, NAND, NOR, AOI, OAI).
 - Check different drive strengths (X1, X2, X4, ...).

Search for a cell

```
grep "cell (INV_X1)" mylib.lib
grep "cell (NAND2_X1)" mylib.lib
grep "cell (AOI221)" mylib.lib
```

Missing cells can stop synthesis or cause suboptimal mapping.
- 7 Check Sequential Cells**
 - Verify presence of flip-flops, latches, and scan cells (e.g., DFF, DFFR, DFFE, LATCH).
 - Check that setup/hold timing arcs are available.

```
grep "cell (DFF)" mylib.lib
grep "cell (LATCH)" mylib.lib
```

Sequential cells are essential for RTL mapping and timing analysis.
- 8 Check Clock Cells**
 - Ensure clock cells (CLKBUF, CLKINV, ICG, etc.) are present.
 - Check different drive strengths.
 - These are required for CTS during PNR.

```
grep "cell (CLKBUF)" mylib.lib
grep "cell (CLKINV)" mylib.lib
grep "cell (ICG)" mylib.lib
```

Missing clock cells will impact CTS and clock tree quality.
- 9 Check Power Cells**
 - Verify presence of special power-related cells (e.g., isolation, level shifter, retention, power switch).
 - Check if low-power variants (LVT/SVT/HVT) are available.

```
grep "ISO" mylib.lib
grep "LSHIFT" mylib.lib
grep "RET" mylib.lib
grep "PWR" mylib.lib
```

Important for low-power and multi-voltage designs.
- 10 Check Timing Model**
 - Identify the timing model used: NLDLM, CCS or ECSM.
 - Ensure it matches the tool requirements.

Example from .lib:

```
delay_model : table_lookup ; # NLDLM
# or
delay_model : ccs ; # CCS
# or
delay_model : ecsm ; # ECSM
```

Advanced nodes usually use CCS. Synthesis may use NLDLM, but signoff requires CCS.
- 11 Check Electrical Limits**
 - Verify max_transition, max_capacitance and max_fanout are defined.
 - These limits are used by synthesis and PNR to maintain signal integrity.

Example from .lib (inside a cell/pin):

```
max_transition : 0.1 ;
max_capacitance : 0.2 ;
max_fanout : 16 ;
```

Ensure the values are reasonable for the technology and match with foundry guidelines.
- 12 Check Consistency with LEF/Netlist**
 - Verify that cell names in .lib match with .lef and netlist.
 - Check pin names and directions.
 - Ensure cell footprints (area) are consistent with LEF.
 - Mismatch can cause PNR errors or incorrect results.

File	Provides	What to Check
.lib	Timing, power, area	Cell names, pin names
.lef	Physical layout	Same cell names, same pins
.v	Netlist (logical)	Same cell names

All three files (.lib, .lef, .v) must be consistent.

Key Takeaways:

- Do these checks before using the .lib in synthesis/ PNR/STA.
- Verify all corners, units, models, cells and consistency with other files.
- A clean and correct .lib setup saves a lot of debug time later in the flow.

"A Small Check Today, A Smoother Tape-out Tomorrow!"

.lib File Example – Part 1: Library Header and Basic Definitions
This part shows the top section of a typical .lib file, which contains library information, units, and default settings.

Real File, Real Format, Used in Industry !

```
tsmc7nm_tt_0p72v_25c.lib

1 library (tsmc7nm_tt_0p72v_25c) {
2   delay_model : ccs;
3   time_unit : "1ps";
4   voltage_unit : "1V";
5   current_unit : "1uA";
6   capacitive_load_unit : "1fF";
7   power_unit : "1mW";
8
9   library_features (
10    "ccs_timing",
11    "ccs_noise",
12    "advanced_leakage_model"
13  );
14
15  nom_process : 1.0;
16  nom_voltage : 0.72;
17  nom_temperature : 25.0;
18  technology : "7nm";
19
20  operating_conditions (tt_0p72v_25c) {
21    process : 1.0;
22    voltage : 0.72;
23    temperature : 25.0;
24  }
25
26  default_operating_conditions : tt_0p72v_25c;
27  default_wire_load_mode : enclosed;
28
29  /* Cell definitions start in next section ... */
30 }
```

- 1 Library Declaration and Units**
 - library (tsmc7nm_tt_0p72v_25c) : Name of the library.
 - delay_model : Specifies the timing model used (e.g., nldm, ccs, ecsm).
 - time_unit, voltage_unit, current_unit, capacitive_load_unit, power_unit : Define the units for all the values in the library.

All timing, power, and capacitance numbers in the .lib are in these units.
- 2 Library Features**
 - Specifies optional features supported by this library.
 - ccs_timing : Composite Current Source (CCS) timing model.
 - ccs_noise : Noise modeling information.
 - advanced_leakage_model : More accurate leakage power modeling.
- 3 Nominal PVT and Technology**
 - nom_process : Nominal process corner (1.0 = typical).
 - nom_voltage : Nominal voltage (in V).
 - nom_temperature : Nominal temperature (in °C).
 - technology : Technology node (e.g., 7nm, 5nm).

These are the typical (TT) conditions for which the library is characterized.
- 4 Operating Conditions**
 - Defines a specific PVT corner used for timing and power data.
 - process : Process corner (1.0 = typical, >1.0 = slow, <1.0 = fast).
 - voltage : Supply voltage in volts.
 - temperature : Temperature in °C.
- 5 Default Settings**
 - default_operating_conditions : Sets the default corner for tools (e.g., STA, synthesis).
 - default_wire_load_mode : Wire load model mode (used in older flows).

Key Takeaway: This is just the beginning of a .lib file. Next, the file will contain cell definitions (e.g., INV, NAND, DFF, etc.) with detailed timing, power, and electrical data.

.lib File Example – Part 2: Cell Definitions (Combinational Cells)
This part shows how a combinational cell (e.g., AND2_X1) is defined with its pins, timing and power data.

Real Library, Real Structure, Used in Industry !

```
tsmc7nm_tt_0p72v_25c.lib (excerpt)

120 cell (AND2_X1) {
121   area : 1.234;
122   cell_leakage_power : 0.00123;
123
124   pin (A) {
125     direction : input;
126     capacitance : 0.0021;
127   }
128 }
```

- 1 Cell Declaration**
 - cell (AND2_X1) : Definition of a standard cell (2-input AND, drive strength X1).
 - area : Physical area of the cell (in μm²).
 - cell_leakage_power : Leakage power of the cell (in W).

Example:

```
cell (AND2_X1) {
  area : 1.234;
  cell_leakage_power : 0.00123;
}
```
- 2 Input Pin Definition (A)**
 - pin (A) : Definition of an input pin.
 - direction : input.
 - capacitance : 0.0021.

Example:

```
pin (A) {
  direction : input;
  capacitance : 0.0021;
}
```

```

127 max_transition : 0.05;
128 max_capacitance : 0.2;
129
130 pin (B) {
131   direction : input;
132   capacitance : 0.0021;
133   max_transition : 0.05;
134   max_capacitance : 0.2;
135 }
136
137 pin (Y) {
138   direction : output;
139   function : "A & B";
140   max_transition : 0.08;
141   max_capacitance : 0.2;
142 }
143
144 timing () {
145   related_pin : "A";
146   cell_rise ("lu_table_template") {
147     index_1 ("0.01, 0.02, 0.05, 0.1"); /* input slew */
148     index_2 ("0.01, 0.02, 0.05, 0.1"); /* output load */
149     values (
150       0.012, 0.013, 0.015, 0.020,
151       0.014, 0.016, 0.019, 0.025,
152       0.018, 0.021, 0.026, 0.034,
153       0.024, 0.029, 0.035, 0.045);
154   }
155   cell_fall ("lu_table_template") {
156     ... /* similar table for fall delay */
157   }
158 }
159
160 internal_power () {
161   related_pin : "A";
162   rise_power ("lu_table_template") { ... }
163   fall_power ("lu_table_template") { ... }
164 }
165
166 } /* end of cell NAND2_X1 */

```

3 Input Pin Definition (B)

- direction : input → this is an input pin.
- capacitance : Input pin capacitance (used for load calculation).
- max_transition : Maximum allowed input slew.
- max_capacitance : Maximum allowed load on this pin.

Example:

```
pin (B) {
  direction : input;
  capacitance : 0.0021;
  max_transition : 0.05;
  max_capacitance : 0.2;
}
```

Similar to pin A. Each input pin has electrical limits and capacitance.

4 Output Pin and Timing Information (Y)

- direction : output → this is an output pin.
- function : Boolean function of the cell ("A & B").
- max_transition : Maximum allowed output transition.
- max_capacitance : Maximum allowed load on output.
- timing : Contains delay and transition tables for rise and fall.
- related_pin : Input pin with respect to which timing is defined.
- cell_rise / cell_fall : 2D lookup tables (LUTs) with input slew (index_1) and output load (index_2).

Example: cell_rise LUT (delay in ns)

Input Slew (ns)	0.01	0.02	0.05	0.1
Output Load (pF)	0.01	0.012	0.013	0.015
0.02	0.014	0.016	0.019	0.025
0.05	0.018	0.021	0.026	0.034
0.1	0.024	0.029	0.035	0.045

5 Internal Power Information

- internal_power : Defines dynamic (switching) power.
- related_pin : Input pin causing the switching.
- rise_power / fall_power : 2D tables for power (in nW).

Example:

```
internal_power () {
  related_pin : "A";
  rise_power ("lu_table_template") { ... }
  fall_power ("lu_table_template") { ... }
}
```

Key Takeaways:

- Each cell in .lib contains pin definitions, timing tables and power tables.
- Timing and power are characterized for different input slews and output loads.
- These tables are used by synthesis, PNR and STA to get accurate delay and power estimates.

Real Numbers, Real Data, Real Silicon Design!

.lib File Example – Part 3: Timing, Power and Constraints inside a Cell

This part shows the detailed timing, power and constraint information inside a cell (e.g., NAND2_X1).

Same File, Real Content, Real Chips!

```

280 cell (NAND2_X1) {
281   area : 1.234;
282   cell_leakage_power : 0.00123;
283 }
284
285 pin (A) {
286   direction : input;
287   capacitance : 0.0021;
288   max_transition : 0.05;
289   max_capacitance : 0.2;
290 }
291
292 timing () {
293   related_pin : "B";
294   timing_type : combinational;
295   cell_rise ("lu_table_template") {
296     index_1 ("0.01, 0.02, 0.05, 0.1"); /* input slew */
297     index_2 ("0.01, 0.02, 0.05, 0.1"); /* output load */
298     values (
299       0.012, 0.014, 0.018, 0.025, \
300       0.013, 0.016, 0.021, 0.028, \
301       0.015, 0.019, 0.025, 0.034, \
302       0.020, 0.024, 0.031, 0.041);
303   }
304   cell_fall ("lu_table_template") {
305     ... /* similar table for fall delay */
306   }
307 }
308
309 internal_power () {
310   related_pin : "A";
311   rise_power ("lu_table_template") {
312     index_1 ("0.01, 0.02, 0.05, 0.1");
313     index_2 ("0.01, 0.02, 0.05, 0.1");
314     values ("0.001, 0.002, 0.003, 0.005, ...");
315   }
316   fall_power ("lu_table_template") {
317     ...
318   }
319 }
320
321 pin (Y) {
322   direction : output;
323   function : "(A & B)";
324   max_transition : 0.08;
325   max_capacitance : 0.2;
326   min_capacitance : 0.0;
327 }
328
329 } /* end of cell NAND2_X1 */

```

1 Cell Header

- Defines the cell name and basic information.
- area : Physical area of the cell (in μm^2).
- cell_leakage_power : Leakage power (in W) at the given PVT corner.

Example:

```
cell (NAND2_X1) {
  area : 1.234;
  cell_leakage_power : 0.00123;
}
```

2 Input Pin Definition

- direction : Specifies input or output.
- capacitance : Input pin capacitance (used for load calculation).
- max_transition : Maximum allowed input slew.
- max_capacitance : Maximum allowed load on the input pin.

Example:

```
pin (A) {
  direction : input;
  capacitance : 0.0021;
  max_transition : 0.05;
  max_capacitance : 0.2;
}
```

3 Timing Information (Delay Tables)

- related_pin : The other pin in the timing path (A → Y through B).
- timing_type : Type of timing (combinational, rising_edge, etc.).
- cell_rise / cell_fall : 2D lookup tables (LUTs) for delay (in ns).
- index_1 : Input slew values (in ns).
- index_2 : Output load values (in pF or fF depending on units).
- values : Delay values for each (slew, load) pair.

Example: cell_rise delay table (ns)

Input Slew (ns)	0.01	0.02	0.05	0.1
Output Load (pF)	0.01	0.012	0.014	0.018
0.02	0.013	0.016	0.021	0.028
0.05	0.015	0.019	0.026	0.034
0.1	0.020	0.024	0.031	0.041

4 Internal Power Information

- related_pin : Input pin that causes the switching.
- rise_power / fall_power : 2D tables for dynamic power (in nW).
- Values represent the switching power consumed during 0→1 (rise) and 1→0 (fall) transitions for different slews and loads.

Example: rise_power table (nW)

Input Slew (ns)	0.01	0.02	0.05	0.1
Output Load (pF)	0.001	0.002	0.003	0.005
0.02	0.0015	0.0025	0.004	0.006
0.05	0.002	0.0035	0.005	0.008
0.1	0.003	0.0045	0.007	0.011

5 Output Pin and Constraints

- direction : Output pin.
- function : Boolean function of the cell.
- max_transition : Maximum allowed output slew.
- max_capacitance : Maximum allowed load.
- min_capacitance : Minimum allowed load (if specified).

Example:

```
pin (Y) {
  direction : output;
  function : "(A & B)";
  max_transition : 0.08;
  max_capacitance : 0.2;
  min_capacitance : 0.0;
}
```

Key Takeaways:

- A .lib file contains detailed cell-level timing, power and constraint information.
- These tables are used by synthesis, PNR and STA for accurate delay, power and SI analysis.
- Understanding .lib structure helps debug timing and constraint issues in the flow.

Same Data, Better Understanding, Better Designs!